

Teoria dos Grafos

Rodrigo Machado
rma@inf.ufrgs.br



Instituto de Informática
Universidade Federal do Rio Grande do Sul
Porto Alegre, Brasil
<http://www.inf.ufrgs.br>

Conteúdo

Grafos simples, incidência, adjacência
Isomorfismo, topologias comuns
Graus
Passeios
Conectividade
Operações sobre grafos
Grafos cíclicos, árvores e florestas
Grafos bipartidos e k-partidos
Pseudografos
Dígrafos
Grafos e dígrafos valorados

Busca em profundidade e busca em largura
Algoritmo de Dijkstra
Grafos eulerianos
Grafos hamiltonianos
Problema do caixeiro viajante
Árvores geradoras mínimas
Algoritmos de Prim e Kruskal
Grafos planares
Coloração de grafos
Enumeração de árvores
Coberturas de vértices
Emparelhamentos

Conteúdo

Grafos simples, incidência, adjacência

Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Grafos simples

Definição: um **grafo simples** é uma tupla (V, E) onde

- V é um conjunto não-vazio de vértices.
- E é um conjunto de subconjuntos de V com exatos dois elementos (chamados arestas).

Grafos simples

Definição: um **grafo simples** é uma tupla (V, E) onde

- V é um conjunto não-vazio de vértices.
- E é um conjunto de subconjuntos de V com exatos dois elementos (chamados arestas).

Exemplo: $G_1 = (\{A, B, C, D, E, F\}, \{\{A, B\}, \{B, D\}, \{B, E\}, \{D, E\}, \{E, F\}\})$

Grafos simples

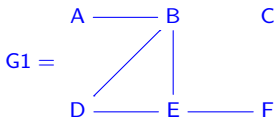
Definição: um **grafo simples** é uma tupla (V, E) onde

- V é um conjunto não-vazio de vértices.
- E é um conjunto de subconjuntos de V com exatos dois elementos (chamados arestas).

Exemplo: $G1 = (\{A, B, C, D, E, F\}, \{\{A, B\}, \{B, D\}, \{B, E\}, \{D, E\}, \{E, F\}\})$

Grafos são comumente *desenhados* como diagramas onde vértices são pontos e arestas são linhas conectando dois vértices distintos.

Exemplo:



Grafos simples: contagem básica

Considere um conjunto V de tamanho n .

Grafos simples: contagem básica

Considere um conjunto V de tamanho n .

Pergunta: se formos construir um grafo simples usando os elementos de V como nodos, qual o número máximo de arestas que podemos inserir?

Grafos simples: contagem básica

Considere um conjunto V de tamanho n .

Pergunta: se formos construir um grafo simples usando os elementos de V como nodos, qual o número máximo de arestas que podemos inserir?

Pergunta: quantos grafos distintos podemos construir sobre o conjunto de vértices V ?

Relação de incidência

Definição: relação de incidência de G (sobre $E \times V$) relaciona *arestas* com os *vértices* que elas conectam.

Relação de incidência

Definição: relação de incidência de G (sobre $E \times V$) relaciona *arestas* com os *vértices* que elas conectam.

Definição: matriz de incidência: representação matricial da relação de incidência de um grafo simples.

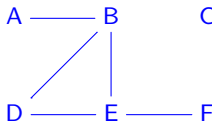
Relação de incidência

Definição: relação de incidência de G (sobre $E \times V$) relaciona *arestas* com os *vértices* que elas conectam.

Definição: matriz de incidência: representação matricial da relação de incidência de um grafo simples.

Exemplo: matriz de incidência de G_1

	A	B	C	D	E	F
$\{A, B\}$	1	1	0	0	0	0
$\{B, D\}$	0	1	0	1	0	0
$\{B, E\}$	0	1	0	0	1	0
$\{D, E\}$	0	0	0	1	1	0
$\{E, F\}$	0	0	0	0	1	1



Relação de adjacência (vértices)

Definição: relação de adjacência de vértices de G (sobre $V \times V$) relaciona vértices conectados por arestas.

Relação de adjacência (vértices)

Definição: relação de adjacência de vértices de G (sobre $V \times V$) relaciona vértices conectados por arestas.

Definição: matriz de adjacência de vértices: representação matricial da relação de adjacência de um grafo simples.

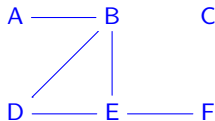
Relação de adjacência (vértices)

Definição: relação de adjacência de vértices de G (sobre $V \times V$) relaciona vértices conectados por arestas.

Definição: matriz de adjacência de vértices: representação matricial da relação de adjacência de um grafo simples.

Exemplo: matriz de adjacência (vértices) de G_1

	A	B	C	D	E	F
A	0	1	0	0	0	0
B	1	0	0	1	1	0
C	0	0	0	0	0	0
D	0	1	0	0	1	0
E	0	1	0	1	0	1
F	0	0	0	0	1	0



Relação de adjacência (arestas)

Definição: relação de adjacência de arestas de G (sobre $E \times E$) relaciona arestas que incidem sobre um único vértice comum.

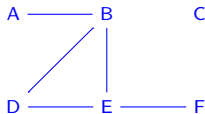
Relação de adjacência (arestas)

Definição: relação de adjacência de arestas de G (sobre $E \times E$) relaciona arestas que incidem sobre um único vértice comum.

Definição: matriz de adjacência: representação matricial da relação de adjacência.

Exemplo: matriz de adjacência (arestas) de G_1

	$\{A, B\}$	$\{B, D\}$	$\{B, E\}$	$\{D, E\}$	$\{E, F\}$
$\{A, B\}$	0	1	1	0	0
$\{B, D\}$	1	0	1	1	0
$\{B, E\}$	1	1	0	1	1
$\{D, E\}$	1	1	1	0	1
$\{E, F\}$	0	0	1	1	0



Propriedades da relação de adjacência

Uma *relação de adjacência* R sobre $U \times U$ é:

- *irreflexiva*:

$$\forall a \in U, \quad (a, a) \notin R$$

Propriedades da relação de adjacência

Uma *relação de adjacência* R sobre $U \times U$ é:

- *irreflexiva*:

$$\forall a \in U, \quad (a, a) \notin R$$

- *simétrica*:

$$\forall a, b \in U, \quad (a, b) \in R \Rightarrow (b, a) \in R$$

Propriedades da relação de adjacência

Uma *relação de adjacência* R sobre $U \times U$ é:

- *irreflexiva*:

$$\forall a \in U, \quad (a, a) \notin R$$

- *simétrica*:

$$\forall a, b \in U, \quad (a, b) \in R \Rightarrow (b, a) \in R$$

Nota: U pode ser V (nodos) ou E (arestas) de um *grafo simples*.

Armazenamento de grafos em memória

Grafos simples podem ser armazenados de várias formas em computadores:

- matriz de incidência
- matriz de adjacência de vértices
- lista encadeada de incidência/adjacência

Armazenamento de grafos em memória

Grafos simples podem ser armazenados de várias formas em computadores:

- matriz de incidência
- matriz de adjacência de vértices
- lista encadeada de incidência/adjacência

Listas encadeadas são mais adequadas quando o número de arestas é pequeno em relação ao número de vértices (matriz de adjacência esparsa).

Armazenamento de grafos em memória

Grafos simples podem ser armazenados de várias formas em computadores:

- matriz de incidência
- matriz de adjacência de vértices
- lista encadeada de incidência/adjacência

Listas encadeadas são mais adequadas quando o número de arestas é pequeno em relação ao número de vértices (matriz de adjacência esparsa).

Contudo, o custo médio de acesso à informação de adjacência em uma lista cresce com o número de vértices.

Exemplo: grafos em Racket

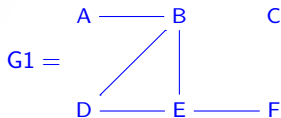
Na linguagem Racket é simples representar grafos através de listas de adjacência.

Definição de dados para grafo (não necessariamente grafo simples):

```
; Uma adjacência é uma lista
; (list s1 s2 ... sn)
; onde
; s1      : símbolo  representa um nodo no grafo
; s2 a sn : símbolo  representam os nodos adjacentes a s1

; Um grafo é uma lista de adjacências
; (list a1 a2 ... an)
; onde todo símbolo referenciado nas adjacências a1 ... an
; ocorre como primeiro elemento em uma única adjacência ak
```


Exemplo: grafos em Racket



Matrix de adjacência:

	A	B	C	D	E	F
A	0	1	0	0	0	0
B	1	0	0	1	1	0
C	0	0	0	0	0	0
D	0	1	0	0	1	0
E	0	1	0	1	0	1
F	0	0	0	0	1	0

Código Racket (lista de adjacências):

```
(define G1
  (list
    (list 'A 'B)
    (list 'B 'A 'D 'E)
    (list 'C)
    (list 'D 'B 'E)
    (list 'E 'B 'D 'F)
    (list 'F 'E)
  )
)
```

Representação gráfica

Nota: um mesmo grafo pode ser *desenhado* de muitas formas diferentes.

Contudo, o jeito de desenhá-lo não o modifica, visto que o grafo é um estrutura algébrica que representa somente conexão entre nodos.

Exemplo:



Conteúdo

Grafos simples, incidência, adjacência

Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Subgrafo

Definição: dois grafos $G = (V, E)$ e $G' = (V', E')$ são iguais sss $V = V'$ e $E = E'$.

Subgrafo

Definição: dois grafos $G = (V, E)$ e $G' = (V', E')$ são iguais sss $V = V'$ e $E = E'$.

Definição: um grafo $G = (V, E)$ é **subgrafo** de $G' = (V', E')$, escrito $G \subseteq G'$, sss $V \subseteq V'$ e $E \subseteq E'$. O grafo G' é chamado **sobregrafo** de G .

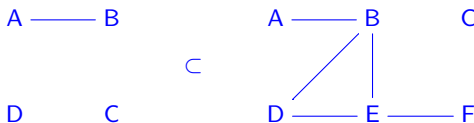
Subgrafo

Definição: dois grafos $G = (V, E)$ e $G' = (V', E')$ são iguais sss $V = V'$ e $E = E'$.

Definição: um grafo $G = (V, E)$ é **subgrafo** de $G' = (V', E')$, escrito $G \subseteq G'$, sss $V \subseteq V'$ e $E \subseteq E'$. O grafo G' é chamado **sobregrafo** de G .

Definição: um grafo G é **subgrafo próprio** de G' , escrito $G \subset G'$, sss $G \subseteq G$ e $G \neq G'$.

Exemplo:



Subgrafo induzido

Definição: Seja $G = (V, E)$ um grafo e $V' \subseteq V$ um subconjunto de vértices de G . O subgrafo G' induzido por V' é definido por:

$$G' = (V', \{ \{a, b\} \mid \{a, b\} \in E \wedge a \in V' \wedge b \in V' \})$$

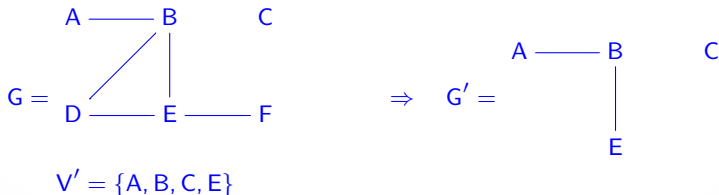
Subgrafo induzido

Definição: Seja $G = (V, E)$ um grafo e $V' \subseteq V$ um subconjunto de vértices de G . O **subgrafo G' induzido por V'** é definido por:

$$G' = (V', \{ \{a, b\} \mid \{a, b\} \in E \wedge a \in V' \wedge b \in V' \})$$

Intuição: o subgrafo induzido por um conjunto de vértices contém todas (e somente) as arestas do grafo original sobre os vértices selecionados.

Exemplo:



Isomorfismo

Definição: um **isomorfismo** entre dois grafos $G = (V, E)$ e $G' = (V', E')$ é uma *função bijetora* $f : V \rightarrow V'$ entre os vértices dos dois grafos tal que

$$\{u, v\} \in E \iff \{f(u), f(v)\} \in E'$$

Isomorfismo

Definição: um **isomorfismo** entre dois grafos $G = (V, E)$ e $G' = (V', E')$ é uma *função bijetora* $f : V \rightarrow V'$ entre os vértices dos dois grafos tal que

$$\{u, v\} \in E \iff \{f(u), f(v)\} \in E'$$

Definição: um isomorfismo de G para si mesmo é chamado **automorfismo**.

Isomorfismo

Definição: um **isomorfismo** entre dois grafos $G = (V, E)$ e $G' = (V', E')$ é uma *função bijetora* $f : V \rightarrow V'$ entre os vértices dos dois grafos tal que

$$\{u, v\} \in E \iff \{f(u), f(v)\} \in E'$$

Definição: um isomorfismo de G para si mesmo é chamado **automorfismo**.

A existência de algum isomorfismo entre dois grafos significa que eles têm exatamente *a mesma forma*, apesar de poderem ter nomes de nodos e arestas distintos, ou serem desenhados de formas diferentes.

Isomorfismo (2)

Definição: dois grafos G e G' são **isomorfos**, denotado $G \cong G'$, sss existe um isomorfismo entre eles.

Isomorfismo (2)

Definição: dois grafos G e G' são **isomorfos**, denotado $G \cong G'$, sss existe um isomorfismo entre eles.

Nota: pode haver dois ou mais isomorfismos distintos entre dois grafos.

Isomorfismo (2)

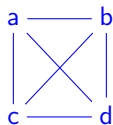
Definição: dois grafos G e G' são **isomorfos**, denotado $G \cong G'$, sss existe um isomorfismo entre eles.

Nota: pode haver dois ou mais isomorfismos distintos entre dois grafos.

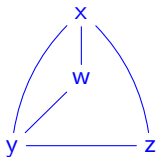
Nota: a relação de isomorfismo (dois grafos são relacionados sss são isomorfos) é uma **relação de equivalência**.

Isomorfismo: exercício

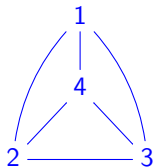
Exercício:



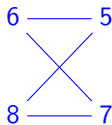
G1



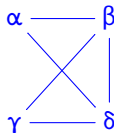
G2



G3



G4

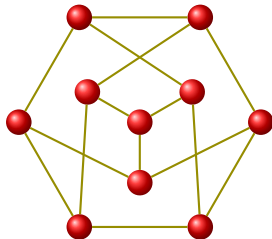
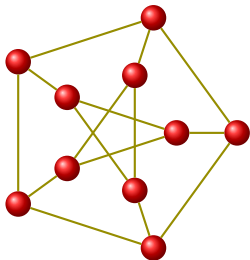


G5

1. Quais grafos acima são isomorfos entre si?
2. Para os grafos isomorfos: quantos isomorfismos distintos há entre eles?
3. Apresente algum automorfismos de G2.

Isomorfismo: exemplo

Os dois grafos abaixo, apesar de aparentemente possuírem formatos distintos, são na verdade isomorfos.



Nota: esse grafo é chamado **grafo de Petersen**.

Grafos abstratos

Podemos enxergar grafos de duas formas:

- concreta: nomes dos nodos e arestas importam, assim como topologia (formato das interconexões).
- abstrata: somente topologia importa.

Grafos abstratos

Podemos enxergar grafos de duas formas:

- concreta: nomes dos nodos e arestas importam, assim como topologia (formato das interconexões).
- abstrata: somente topologia importa.

Definição: um **grafo abstrato** é uma classe de equivalência de grafos isomorfos.

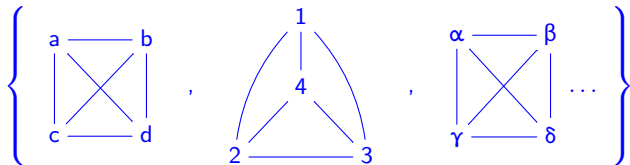
Grafos abstratos

Podemos enxergar grafos de duas formas:

- concreta: nomes dos nodos e arestas importam, assim como topologia (formato das interconexões).
- abstrata: somente topologia importa.

Definição: um **grafo abstrato** é uma classe de equivalência de grafos isomorfos.

Exemplo: grafo abstrato de quatro vértices, totalmente conectado =



Subgrafo a menos de isomorfismo

Habitualmente estamos interessados em grafos abstratos, e não nas representações concretas.

Podemos estender conceitos como **subgrafo** para o caso abstrato.

Subgrafo a menos de isomorfismo

Habitualmente estamos interessados em grafos abstratos, e não nas representações concretas.

Podemos estender conceitos como **subgrafo** para o caso abstrato.

Definição: G é subgrafo de G' a menos de isomorfismo sss existe $H \subseteq G'$ tal que $G \cong H$.

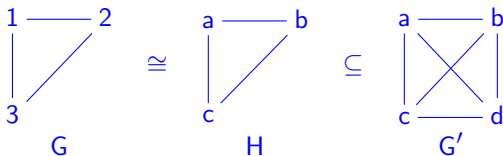
Subgrafo a menos de isomorfismo

Habitualmente estamos interessados em grafos abstratos, e não nas representações concretas.

Podemos estender conceitos como **subgrafo** para o caso abstrato.

Definição: G é **subgrafo de G' a menos de isomorfismo** sss existe $H \subseteq G'$ tal que $G \cong H$.

Exemplo:



Famílias comuns de grafos simples

Algumas formas para grafos representam padrões comuns, e portanto recebem nomes especiais (normalmente parametrizados pelo número de vértices).

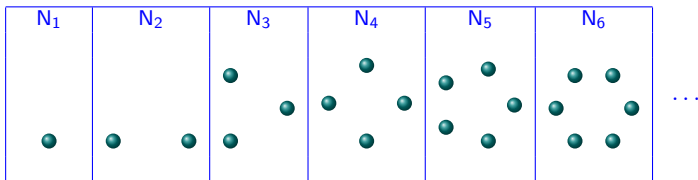
Veremos os seguintes *formatos* de grafo:

- nulo (ou discreto)
- completo
- caminho
- círculo
- grade
- estrela
- roda
- k-cubo

Nota: cada topologia com um dado número de nodos representa um grafo abstrato distinto.

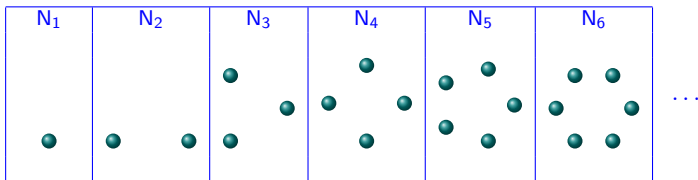
Grafos nulos e completos

Grafo nulo: N_n (sem arestas)

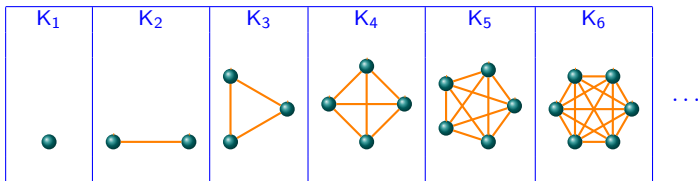


Grafos nulos e completos

Grafo nulo: N_n (sem arestas)



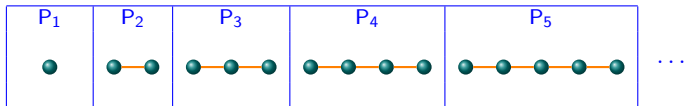
Grafo completo: K_n (todas as arestas possíveis)



Curiosidade: K vem da palavra “komplett” em alemão.

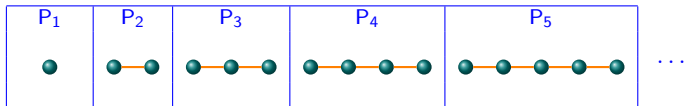
Grafos caminho e ciclo

Grafo caminho: P_n ("path")

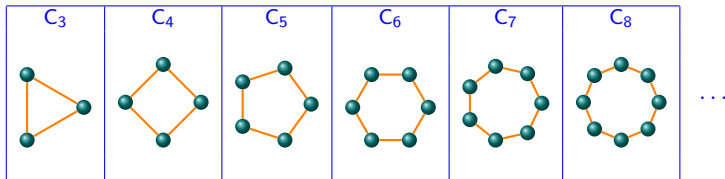


Grafos caminho e ciclo

Grafo caminho: P_n ("path")

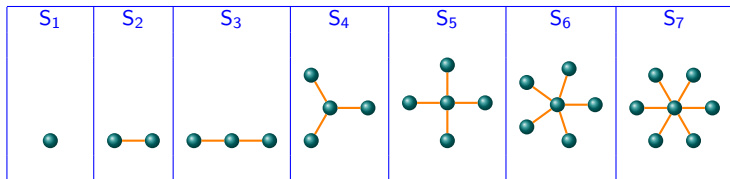


Grafo ciclo: C_n (para $n \geq 3$)



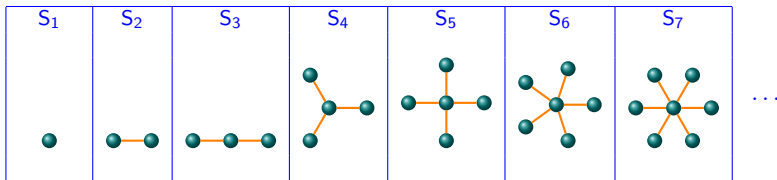
Grafos estrela e roda

Grafo estrela: S_n ("star")

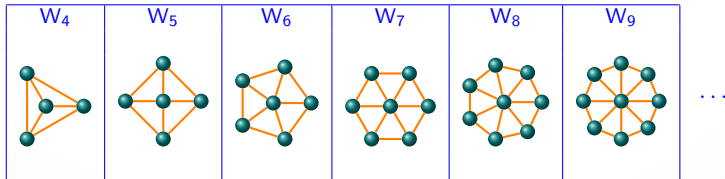


Grafos estrela e roda

Grafo estrela: S_n ("star")

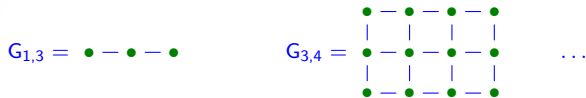


Grafo roda: W_n ("wheel") $n \geq 4$



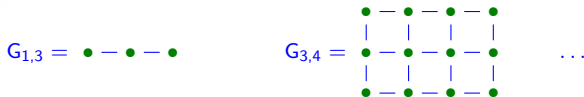
Grafos grade e k-cubo

Grafo grade: $G_{m,n}$ (m linhas, n colunas).

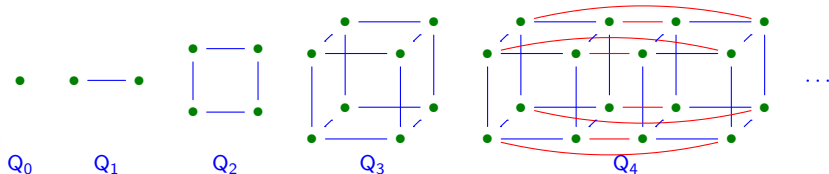


Grafos grade e k-cubo

Grafo grade: $G_{m,n}$ (m linhas, n colunas).



Grafo k-cubo: Q_k



Nota: ao contrário das outras famílias, onde n representa o número de vértices, k representa as dimensões do cubo.

Famílias comuns de grafos simples: exercício

Exercício: Determine fórmulas de contagem do número de arestas a partir do número de vértices para as seguintes famílias de grafos:

- nulo (ou discreto)
- completo
- caminho
- círculo
- grade
- estrela
- roda
- k-cubo

Conteúdo

Grafos simples, incidência, adjacência
Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

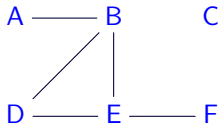
Coberturas de vértices

Emparelhamentos

Grau de um nodo

Definição: o **grau** de um nodo n , escrito $\deg(n)$, é o número de arestas que incidem sobre n .

Exemplo:



- $\deg(A) = 1$
- $\deg(B) = 3$
- $\deg(C) = 0$
- $\deg(D) = 2$
- $\deg(E) = 3$
- $\deg(F) = 1$

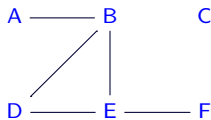
Nota: $\deg(n)$ é definido somente quando o número de arestas incidentes sobre n é finito.

Grau de um nodo e matriz de incidência

Em um grafo simples, podemos calcular o grau de um nodo somando a coluna correspondente ao nodo na matriz de incidência do grafo.

Exemplo:

	A	B	C	D	E	F
{A, B}	1	1	0	0	0	0
{B, D}	0	1	0	1	0	0
{B, E}	0	1	0	0	1	0
{D, E}	0	0	0	1	1	0
{E, F}	0	0	0	0	1	1
deg	1	3	0	2	3	1



Lema: (*handshaking lemma*) em um grafo simples $G = (V, E)$

$$\sum_{v \in V} \deg(v) = 2|E|$$

Demonstração: soma total da matriz de incidência por linhas vs. por colunas

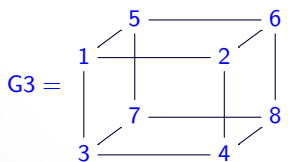
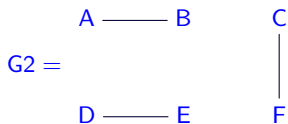
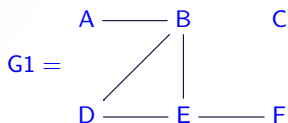
Graus de um grafo

Definição: o **grau mínimo** de um grafo G , denotado $\delta(G)$, é o menor grau de nodo do grafo.

Definição: o **grau máximo** de um grafo G , denotado $\Delta(G)$, é o maior grau de nodo do grafo.

Definição: um grafo é **k -regular**, ou simplesmente regular, quando $\delta(G) = \Delta(G) = k$ (todos os nodos têm o mesmo grau k).

Graus de um grafo: exemplos



- $\delta(G1) = 0$
- $\Delta(G1) = 3$
- G1 não é regular
- $\delta(G2) = 1$
- $\Delta(G2) = 1$
- G2 é 1-regular
- $\delta(G3) = 3$
- $\Delta(G3) = 3$
- G3 é 3-regular

Conteúdo

Grafos simples, incidência, adjacência
Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Passeio

Definição: Um **passeio** (walk) p entre nodos x e y , denotado $p(x, y)$, é uma sequência alternada

$$p(x, y) = v_0, e_1, v_1, e_2, v_2 \dots e_n, v_n$$

de vértices e arestas no qual

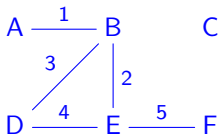
- $x = v_0$ e $y = v_n$
- há ao menos uma aresta
- toda aresta está situada entre dois nodos sobre os quais incide

Definição: o **tamanho** do passeio, denotado $|p|$, é a quantidade de arestas presentes (contando repetições).

Nota: no caso de grafos simples o passeio é determinado unicamente pelos vértices, portanto podemos escrever simplesmente $p(v_1, v_n) = v_1, v_2, \dots v_n$. Outros tipos de grafos (definidos a seguir) requerem que se *nomeie* explicitamente as arestas.

Passeio: exemplo

Grafo exemplo (arestas nomeadas):



Exemplos de passeios:

- $p_1(A, B) = A, 1, B$ $|p_1| = 1$
- $p_2(A, E) = A, 1, B, 2, E$ $|p_2| = 2$
- $p_3(B, A) = B, 2, E, 4, D, 3, B, 2, E, 4, D, 3, B, 1, A$ $|p_3| = 7$

Tipos de passeio

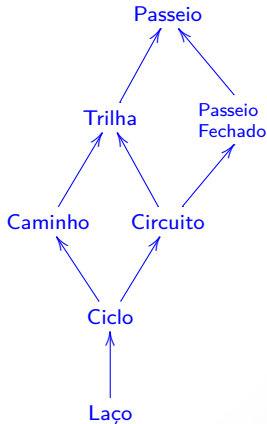
Definição: um passeio $p(x, y)$ é **fechado** quando $x = y$.

Definição: uma **trilha** (trail) é um *passeio* sem arestas repetidas.

Definição: um **caminho** (path) é uma *trilha* $p(x, y)$ sem nodos repetidos (com exceção de x e y , que podem ser iguais).

Definição: nomes especiais para alguns passeios:

- **circuito** (circuit): trilha fechada
- **ciclo** (cycle): caminho fechado
- **laço** (loop): caminho fechado de tamanho 1



Propriedades de passeios

Lema: de todo passeio $p(x, y)$ podemos extrair um caminho $c(x, y)$.

Demonstração: por indução no tamanho de p . **Hipótese Indutiva:** de todo passeio $p'(a, b)$ onde $|p'| < |p|$ podemos extrair um caminho entre a e b .

1. **base:** $p(x, y) = x, y$. Neste caso, p é o próprio caminho.
2. **passo de indução:** $p(x, y) = p', y$, sendo $p'(x, y)$ um sub-passeio. Como $|p'| < |p|$, pela H.I. temos que existe caminho $c'(x, y)$. Duas possibilidades:
 - $y \notin (c' - \{x\})$. Neste caso, o passeio $c = c', y$ não forma ciclos internos e portanto é o caminho desejado.
 - $y \in (c' - \{x\})$. Então c', y forma um ciclo interno. Portanto, há uma primeira ocorrência de y dentro do caminho $c'(x, y)$:

$$c' = x, \dots, y, \dots, y$$

Tome como caminho desejado a subsequência $x, \dots, y$ até a primeira ocorrência de y em c' .

Conteúdo

Grafos simples, incidência, adjacência
Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Conectividade de nós

Podemos estender a noção de adjacência (nodos ligados por arestas) para *conectividade* (nodos ligados por rotas).

Definição: dois nós a e b estão **conectados** (denotado $a \leftrightarrow b$) sss

- $a = b$, ou
- existe uma *rota* entre a e b

Conectividade de nós

Podemos estender a noção de adjacência (nodos ligados por arestas) para *conectividade* (nodos ligados por rotas).

Definição: dois nós a e b estão **conectados** (denotado $a \leftrightarrow b$) sss

- $a = b$, ou
- existe uma *rota* entre a e b

Definição: a **relação de conectividade** determina que dois vértices estão relacionados sss estão conectados.

Conectividade de nós

Podemos estender a noção de adjacência (nodos ligados por arestas) para *conectividade* (nodos ligados por rotas).

Definição: dois nós a e b estão **conectados** (denotado $a \rightsquigarrow b$) sss

- $a = b$, ou
- existe uma *rota* entre a e b

Definição: a **relação de conectividade** determina que dois vértices estão relacionados sss estão conectados.

Lema: em grafos simples, a relação de conectividade é uma *relação de equivalência* sobre o conjunto de vértices.

Demonstração: Exercício!

Distância entre nodos

Seja $G = (V, E)$ um grafo simples.

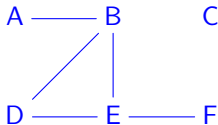
Definição: a **distância** entre dois nós a e b , denotada $d(a, b)$, é definida por:

$$d : V \times V \rightarrow \mathbb{N} \cup \{\infty\}$$

$$d(a, b) = \begin{cases} 0 & \text{se } a = b \\ \text{menor comprimento de passeio entre } a \text{ e } b & \text{se } (a \neq b) \wedge (a \leftrightarrow b) \\ \infty & \text{se } a \not\leftrightarrow b \end{cases}$$

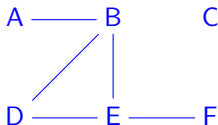
Conectividade e distância: exemplo

Exemplo:



Conectividade e distância: exemplo

Exemplo:

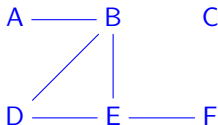


Conectividade:

- A e A são conectados (iguais).

Conectividade e distância: exemplo

Exemplo:

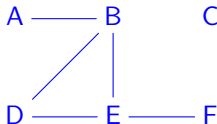


Conectividade:

- A e A são conectados (iguais).
- A e F são conectados (caminhos ABDEF e ABEF).

Conectividade e distância: exemplo

Exemplo:

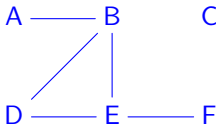


Conectividade:

- A e A são conectados (iguais).
- A e F são conectados (caminhos ABDEF e ABEF).
- E e C não são conectados.

Conectividade e distância: exemplo

Exemplo:



Conectividade:

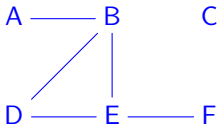
- A e A são conectados (iguais).
- A e F são conectados (caminhos ABDEF e ABEF).
- E e C não são conectados.

Distância:

- $d(A, A) = 0$

Conectividade e distância: exemplo

Exemplo:



Conectividade:

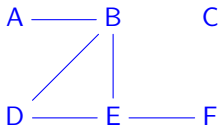
- A e A são conectados (iguais).
- A e F são conectados (caminhos ABDEF e ABEF).
- E e C não são conectados.

Distância:

- $d(A, A) = 0$
- $d(A, B) = 1$

Conectividade e distância: exemplo

Exemplo:



Conectividade:

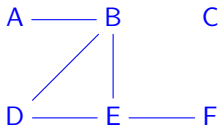
- A e A são conectados (iguais).
- A e F são conectados (caminhos ABDEF e ABEF).
- E e C não são conectados.

Distância:

- $d(A, A) = 0$
- $d(A, B) = 1$
- $d(A, C) = \infty$

Conectividade e distância: exemplo

Exemplo:



Conectividade:

- A e A são conectados (iguais).
- A e F são conectados (caminhos ABDEF e ABEF).
- E e C não são conectados.

Distância:

- $d(A, A) = 0$
- $d(A, B) = 1$
- $d(A, C) = \infty$
- $d(B, F) = 2$

Grafos conexos e desconexos

Definição: um grafo é **conexo** quando temos $a \rightsquigarrow b$ para todos os vértices a e b .

Grafos conexos e desconexos

Definição: um grafo é **conexo** quando temos $a \leftrightarrow b$ para todos os vértices a e b .

Definição: Um grafo é **desconexo** quando existem dois vértices a e b tal que $a \not\leftrightarrow b$.

Grafos conexos e desconexos

Definição: um grafo é **conexo** quando temos $a \leftrightarrow b$ para todos os vértices a e b .

Definição: Um grafo é **desconexo** quando existem dois vértices a e b tal que $a \nleftrightarrow b$.

Definição: H é um **componente conexo** de G sss

- H é subgrafo conexo de G
- H não está propriamente contido em nenhum outro subgrafo conexo de G

Grafos conexos e desconexos

Definição: um grafo é **conexo** quando temos $a \rightsquigarrow b$ para todos os vértices a e b .

Definição: Um grafo é **desconexo** quando existem dois vértices a e b tal que $a \not\rightsquigarrow b$.

Definição: H é um **componente conexo** de G sss

- H é subgrafo conexo de G
- H não está propriamente contido em nenhum outro subgrafo conexo de G

Em outras palavras, um *componente conexo* é um subgrafo conexo *maximal*.

Cliques

Definição: um **clique** de G é um subgrafo $H \subseteq G$ tal que $H \cong K_n$ (para algum $n \geq 1$).

Cliques

Definição: um **clique** de G é um subgrafo $H \subseteq G$ tal que $H \cong K_n$ (para algum $n \geq 1$).

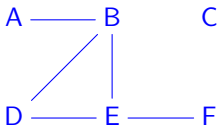
Definição: um clique de G é **maximal** sss não está contido propriamente em nenhum outro clique de G

Cliques

Definição: um **clique** de G é um subgrafo $H \subseteq G$ tal que $H \cong K_n$ (para algum $n \geq 1$).

Definição: um clique de G é **maximal** sss não está contido propriamente em nenhum outro clique de G

Exercício: para o grafo abaixo, determine componentes conexos, cliques e cliques maximais.



Conjuntos independentes

Definição: um **conjunto independente** de $G = (V, E)$ é um subconjunto de V onde nenhum par de vértices é adjacente.

Conjuntos independentes

Definição: um **conjunto independente** de $G = (V, E)$ é um subconjunto de V onde nenhum par de vértices é adjacente.

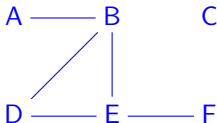
Definição: um conjunto independente de G é **maximal** se não pudermos adicionar nenhum vértice a ele sem introduzir uma aresta.

Conjuntos independentes

Definição: um **conjunto independente** de $G = (V, E)$ é um subconjunto de V onde nenhum par de vértices é adjacente.

Definição: um conjunto independente de G é **maximal** se não pudermos adicionar nenhum vértice a ele sem introduzir uma aresta.

Exercício: para o grafo abaixo, determine os conjuntos independentes e conjuntos independentes maximais.



Conteúdo

Grafos simples, incidência, adjacência
Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Operações sobre grafos

Grafos simples podem ser manipulados de diversas formas:

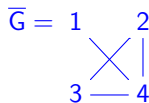
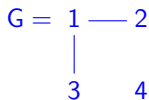
- complemento, união, interseção, união disjunta
- deleção de aresta/nodo
- soma, multiplicação

Complemento de grafos

Definição: Complemento $\overline{G}$ (válido somente para grafos simples)

$$\overline{(V, E)} = (V, \{e \mid e \in 2^V \wedge |e| = 2 \wedge e \notin E\})$$

Exemplo:

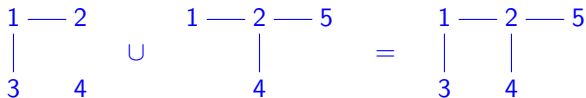


União de grafos

Definição: União $G_1 \cup G_2$

$$(V_1, E_1) \cup (V_2, E_2) = (V_1 \cup V_2, E_1 \cup E_2)$$

Exemplo:

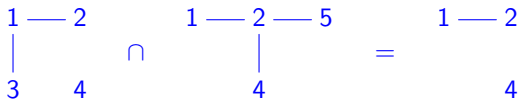


Interseção de grafos

Definição: Interseção $G_1 \cap G_2$

$$(V_1, E_1) \cap (V_2, E_2) = (V_1 \cap V_2, E_1 \cap E_2)$$

Exemplo:

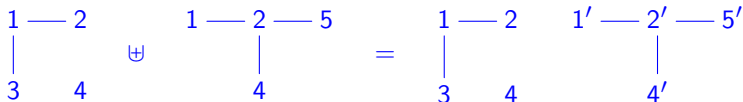


União disjunta

Definição: União disjunta $G_1 \uplus G_2$

$$(V_1, E_1) \uplus (V_2, E_2) = (V_1 \uplus V_2, E_1 \uplus E_2)$$

Exemplo:



Remoção de aresta

Definição: Remoção de aresta $G - e$

$$(V, E) - \{a, b\} = (V, E - \{a, b\})$$

Exemplo:

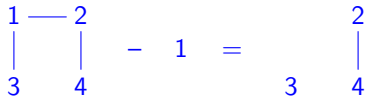
$$\begin{array}{cc} 1 & \text{---} & 2 \\ | & & \\ 3 & & 4 \end{array} - \{1, 2\} = \begin{array}{cc} 1 & 2 \\ | & \\ 3 & 4 \end{array}$$

Remoção de nodo

Definição: Remoção de nodo $G - v$

$$(V, E) - v = (V - \{v\}, E - \{e \mid e \in E \wedge v \in e\})$$

Exemplo:



Nota: todas as arestas incidentes ao nodo deletado também são removidas.

Remoção de subgrafo

Definição: Subtração de grafos

$$G - H = G'$$

onde

$$H = (\{v_1, v_2, \dots, v_n\}, \{e_1, e_2, \dots, e_m\})$$

$$G' = G - e_1 - e_2 - \dots - e_m - v_1 - v_2 - \dots - v_n$$

Exemplo:

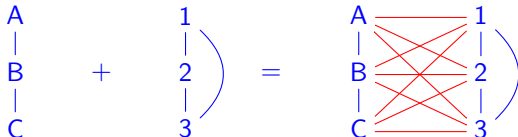
$$\begin{array}{c} 1 \text{ --- } 2 \\ | \quad | \\ 3 \quad 4 \end{array} - \begin{array}{c} 2 \\ | \\ 4 \end{array} = \begin{array}{c} 1 \\ | \\ 3 \end{array}$$

Soma (join)

Definição: Soma (join) $G_1 + G_2$

$$(V_1, E_1) + (V_2, E_2) = (V_1 \uplus V_2, E_1 \uplus E_2 \uplus \{\{a, b\} \mid a \in V_1 \wedge b \in V_2\})$$

Exemplo:

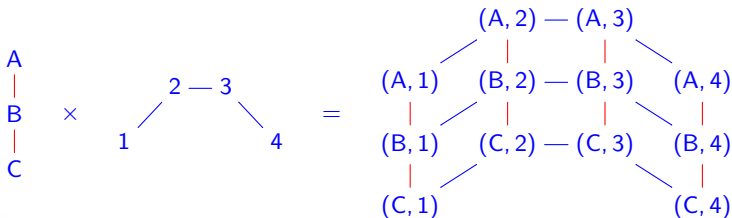


Produto cartesiano

Definição: Produto cartesiano $G_1 \times G_2$

$$(V_1, E_1) \times (V_2, E_2) = \left(V_1 \times V_2, \begin{array}{c} \{ \{(x, a), (x, b)\} \mid x \in V_1 \wedge \{a, b\} \in E_2 \} \\ \cup \\ \{ \{(a, x), (b, x)\} \mid \{a, b\} \in E_1 \wedge x \in V_2 \} \end{array} \right)$$

Exemplo:



Conteúdo

Grafos simples, incidência, adjacência

Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Grafos cíclicos e acíclicos

Definição: um grafo G é **cíclico** sss existe um subgrafo $G' \subseteq G$ tal que $G' \cong C_k$ (para algum $k \geq 3$).

Grafos cíclicos e acíclicos

Definição: um grafo G é **cíclico** sss existe um subgrafo $G' \subseteq G$ tal que $G' \cong C_k$ (para algum $k \geq 3$).

Definição: um grafo é **acíclico** se não for cíclico.

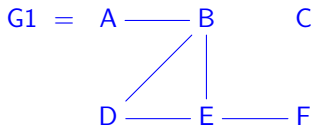
Grafos cíclicos e acíclicos

Definição: um grafo G é **cíclico** sss existe um subgrafo $G' \subseteq G$ tal que $G' \cong C_k$ (para algum $k \geq 3$).

Definição: um grafo é **acíclico** se não for cíclico.

Exemplo:

Grafo cíclico:



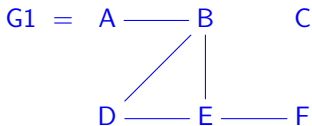
Grafos cíclicos e acíclicos

Definição: um grafo G é **cíclico** sss existe um subgrafo $G' \subseteq G$ tal que $G' \cong C_k$ (para algum $k \geq 3$).

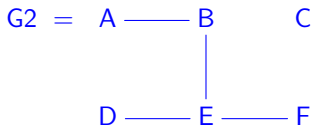
Definição: um grafo é **acíclico** se não for cíclico.

Exemplo:

Grafo cíclico:



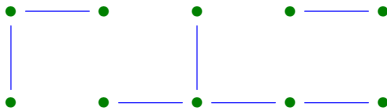
Grafo acíclico:



Árvores e florestas

Definição: um grafo simples é chamado **floresta** se é *acíclico*.

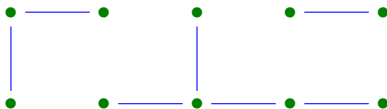
Exemplo:



Árvores e florestas

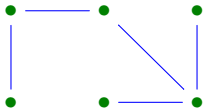
Definição: um grafo simples é chamado **floresta** se é *acíclico*.

Exemplo:



Definição: um grafo simples é chamado **árvore** se é *conexo* e *acíclico*.

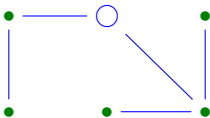
Exemplo:



Árvores enraizadas

Definição: uma **árvore enraizada** é uma tupla (G, v) onde $G = (V, E)$ é uma árvore e $v \in V$ é um nodo especial chamado *raiz*.

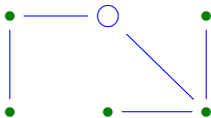
Exemplo:



Árvores enraizadas

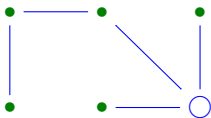
Definição: uma **árvore enraizada** é uma tupla (G, v) onde $G = (V, E)$ é uma *árvore* e $v \in V$ é um nodo especial chamado *raiz*.

Exemplo:



Nota: distintas raízes caracterizam distintas *árvores enraizadas*.

Exemplo: (diferente da árvore enraizada acima)



Caracterização de árvores

As quatro definições abaixo determinam que o grafo G é uma árvore:

1. G é conexo e acíclico
2. G é conexo e possui $n - 1$ arestas
3. G possui $n - 1$ arestas e não há ciclos
4. para dois nodos quaisquer $a, b \in V(G)$, há um único caminho $c(a, b)$ os conectando

Definições distintas porém equivalentes para exatamente o mesmo conceito são denominadas *caracterizações*.

Demonstração:

- $(1) \Rightarrow (2) \wedge (3)$

Caracterização de árvores

As quatro definições abaixo determinam que o grafo G é uma árvore:

1. G é conexo e acíclico
2. G é conexo e possui $n - 1$ arestas
3. G possui $n - 1$ arestas e não há ciclos
4. para dois nodos quaisquer $a, b \in V(G)$, há um único caminho $c(a, b)$ os conectando

Definições distintas porém equivalentes para exatamente o mesmo conceito são denominadas *caracterizações*.

Demonstração:

- $(1) \Rightarrow (2) \wedge (3)$
- $(2) \Rightarrow (1) \wedge (3)$

Caracterização de árvores

As quatro definições abaixo determinam que o grafo G é uma árvore:

1. G é conexo e acíclico
2. G é conexo e possui $n - 1$ arestas
3. G possui $n - 1$ arestas e não há ciclos
4. para dois nodos quaisquer $a, b \in V(G)$, há um único caminho $c(a, b)$ os conectando

Definições distintas porém equivalentes para exatamente o mesmo conceito são denominadas *caracterizações*.

Demonstração:

- $(3) \Rightarrow (1) \wedge (2)$

- $(1) \Rightarrow (2) \wedge (3)$

- $(2) \Rightarrow (1) \wedge (3)$

Caracterização de árvores

As quatro definições abaixo determinam que o grafo G é uma árvore:

1. G é conexo e acíclico
2. G é conexo e possui $n - 1$ arestas
3. G possui $n - 1$ arestas e não há ciclos
4. para dois nodos quaisquer $a, b \in V(G)$, há um único caminho $c(a, b)$ os conectando

Definições distintas porém equivalentes para exatamente o mesmo conceito são denominadas *caracterizações*.

Demonstração:

- $(1) \Rightarrow (2) \wedge (3)$
- $(2) \Rightarrow (1) \wedge (3)$
- $(3) \Rightarrow (1) \wedge (2)$
- $(1) \Rightarrow (4)$

Caracterização de árvores

As quatro definições abaixo determinam que o grafo G é uma árvore:

1. G é conexo e acíclico
2. G é conexo e possui $n - 1$ arestas
3. G possui $n - 1$ arestas e não há ciclos
4. para dois nodos quaisquer $a, b \in V(G)$, há um único caminho $c(a, b)$ os conectando

Definições distintas porém equivalentes para exatamente o mesmo conceito são denominadas *caracterizações*.

Demonstração:

- $(1) \Rightarrow (2) \wedge (3)$
- $(2) \Rightarrow (1) \wedge (3)$
- $(3) \Rightarrow (1) \wedge (2)$
- $(1) \Rightarrow (4)$
- $(4) \Rightarrow (1)$

Conteúdo

Grafos simples, incidência, adjacência

Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

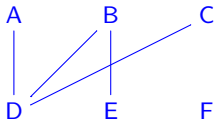
Grafos bipartidos

Um grafo simples $G = (V, E)$ é **bipartido** sss $V = V_1 \cup V_2$ onde

- $V_1 \cap V_2 = \emptyset$ (conjuntos disjuntos)
- V_1 e V_2 são ambos *conjuntos independentes* de vértices de G (não há arestas ligando diretamente dois vértices em V_1 ou dois vértices em V_2)

Intuição: um grafo bipartido é aquele onde toda aresta conecta um vértice em V_1 a um vértice em V_2 .

Exemplo:



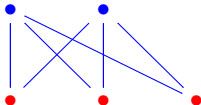
$$\begin{aligned} V &= \{A, B, C, D, E, F\} \\ V_1 &= \{A, B, C\} \\ V_2 &= \{D, E, F\} \end{aligned}$$

Grafos bipartidos completos

Definição: um grafo bipartido é **bipartido completo** quando possui todas as arestas *possíveis* entre os nodos.

Notação: $K_{m,n}$ (onde m e n são as cardinalidades dos conjuntos de vértices)

Exemplo: $K_{2,3}$



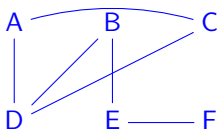
Grafos k-partidos

Um grafo simples $G = (V, E)$ é k -partido sss $V = V_1 \cup V_2 \cup \dots \cup V_k$ onde

- V_1 a V_k são todos disjuntos entre si (par a par)
- V_1 a V_k são todos conjuntos independentes de vértices de G

Intuição: extensão da noção de grafo bipartido para n classes de vértices.

Exemplo: (grafo tripartido)



$$V = \{A, B, C, D, E, F\}$$

$$V_1 = \{A, B\}$$

$$V_2 = \{D, E\}$$

$$V_3 = \{C, F\}$$

Grafos bipartidos: exercício

Exercício: verifique se os grafos com as seguintes topologias são bipartidos ou não.

- nulo
- completo
- caminho
- círculo
- grade
- estrela
- roda
- k-cubo

Caracterização de grafos bipartidos

Lema: todo passeio fechado p de tamanho ímpar contém um ciclo c de tamanho ímpar.

Demonstração: por indução no tamanho de p

- **caso base:** (tamanho 3 em grafos simples). Tome p como c .
- **passo indutivo:** Pela hipótese indutiva, a propriedade vale para todo p' onde $|p'| < |p|$.
Duas possibilidades para p :
 - p não contém vértices repetidos (desconsiderando os pontos iniciais e finais). Nesse caso, tome p como o ciclo ímpar c .
 - p contém uma repetição de vértice em v . Portanto, v divide o caminho fechado em dois subcaminhos fechados q e q' . Como p possui tamanho ímpar, temos que um dos subcaminhos fechados (digamos q) possui tamanho ímpar, sendo que $|q| < |p|$. Pela hipótese indutiva, q possui ciclo ímpar. □

Caracterização de grafos bipartidos (2)

Teorema: (König, 1936) um grafo é bipartido sss não contém nenhum ciclo de tamanho ímpar.

Demonstração: ($\Rightarrow$): se G é bipartido, o conjunto de vértices é divisível em dois conjuntos independentes V_1 e V_2 . Logo, todo passeio alterna entre nodos de V_1 e V_2 . Portanto todo passeio fechado precisa percorrer um número par de passos. Logo, G não contém ciclos de tamanho ímpar.

($\Leftarrow$): G não contém ciclos de tamanho ímpar. Vamos mostrar que todo componente conexo de G pode ter seus vértices divididos em dois conjuntos independentes. Tendo isso, podemos compor uma bipartição de G .

Seja H um componente conexo de G , e u um nodo $u \in V(H)$. Considere a distância $d(u, v)$ para cada $v \in V(H)$, e classifique os nodos de H em dois subconjuntos:

$X = \{v \mid d(u, v) \text{ par}\}$ e $Y = \{v \mid d(u, v) \text{ ímpar}\}$. Se houvesse alguma aresta $e = \{a, b\}$ interna a X ou Y , haveria um passeio de tamanho ímpar $u, \dots, a, b, \dots, u$. Pelo lema do slide anterior, esse passeio teria um ciclo ímpar (contradição). Logo, não há aresta interna a X e a Y , e portanto ambos são conjuntos independentes, onde $X \cup Y = V(H)$. □

Conteúdo

Grafos simples, incidência, adjacência

Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Pseudografos e multigrafos

Definição: um **pseudografo** é uma tripla (V, E, conn) onde

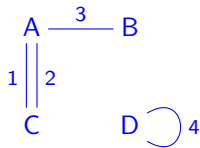
- V é um conjunto de vértices
- E é um conjunto de arestas
- $\text{conn} : E \rightarrow 2^V$ associa cada aresta a um subconjunto de V com cardinalidade 2 ou 1.

Definição: uma aresta $e \in E$ é **endoaresta** (ou laço) sss $|\text{conn}(e)| = 1$.

Definição: um **multigrafo** é um pseudografo (V, E, conn) sem laços (para todo $e \in E$, $|\text{conn}(e)| = 2$).

Pseudografos: exemplo

Exemplo:



$$V = \{A, B, C, D\}$$

$$E = \{1, 2, 3, 4\}$$

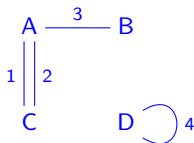
$$\text{conn} = \begin{cases} 1 \mapsto \{A, C\} \\ 2 \mapsto \{A, C\} \\ 3 \mapsto \{A, B\} \\ 4 \mapsto \{D\} \end{cases}$$

Nota: ao contrário de grafos simples, pseudografos admitem arestas paralelas e laços. Multigrafos admitem arestas paralelas mas não admitem laços.

Pseudografos: incidência e grau de nodos

Em grafos gerais, laços são registrados na matriz de incidência com o número 2 ao invés de 1

Exemplo:



	A	B	C	D
1	1	0	1	0
2	1	0	1	0
3	1	1	0	0
4	0	0	0	2

Nota: matriz de incidência deixa de ser representação da relação de incidência, pois permite valores diferentes de 0 e 1.

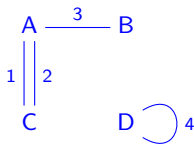
No cálculo do grau de um nodo em um pseudografo, laços pesam 2.

Exemplo: $\deg(D) = 2$

Pseudografos: adjacência

Na matriz de adjacência de vértices, registramos na posição (i, j) a quantidade de arestas entre o i -ésimo e o j -ésimo vértices.

Exemplo:



	A	B	C	D
A	0	1	2	0
B	1	0	0	0
C	2	0	0	0
D	0	0	0	1

Nota: sobre matriz e relação de adjacência em grafos gerais:

- a matriz deixa de ser uma simples representação da relação, pois a primeira possui mais informação (quantidade de arestas paralelas) que a segunda (que só indica existência ou não de aresta).
- ambas (matriz e relação) são simétricas, porém não são irreflexivas (diagonal principal pode ser não-nula).

Pseudografos: passeios e conectividade

Conceitos como passeio, trilhas, etc. são os mesmos em grafos simples e pseudografos.

Nota: no caso de passeios em pseudografos, devemos obrigatoriamente nomear as arestas.

Exemplo:

$$G1 = A \overset{1}{\text{---}} B$$

- Em $G1$, não existe caminho fechado iniciando em A . O passeio $A, 1, B, 1, A$ não é trilha.

$$G2 = A \underset{2}{\text{---}} B$$

- Em $G2$, existe caminho fechado iniciando em A . O passeio $A, 1, B, 2, A$ é trilha e caminho.

Conceitos relacionados à conectividade (cliques, conjuntos independentes, componentes maximais) se mantêm os mesmos.

Pseudografos: ciclos

Definição: um **pseudografo** $G = (V, E)$ é **cíclico** se houver um caminho $c(v, v)$ para algum nodo $v \in V$. Caso contrário, G é acíclico.

Em pseudografos há mais possibilidades de haver ciclos do que em grafos simples:

Exemplo: (pseudografos cíclicos)

Ciclo de tamanho 2:



Ciclo de tamanho 1:



Conteúdo

Grafos simples, incidência, adjacência

Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Dígrafos

Definição: um **dígrafo** (ou grafo dirigido) é uma tripla (V, E, conn) onde

- V é um conjunto de vértices
- E é um conjunto de arestas
- $\text{conn} : E \rightarrow V \times V$ associa cada aresta e a um par ordenado (a, b) de vértices incidentes, onde
 - a representa a *origem* de e . Notação: $\text{src}(e) = a$.
 - b representa o *destino* de e . Notação: $\text{tgt}(e) = b$.

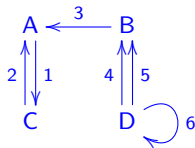
Desenhamos uma aresta e na qual $\text{conn}(e) = (a, b)$ como uma seta

$$a \xrightarrow{e} b$$

de a para b .

Dígrafos: exemplo

Exemplo:



$$V = \{A, B, C, D\}$$

$$E = \{1, 2, 3, 4, 5, 6\}$$

$$\text{conn} = \begin{cases} 1 \mapsto (A, C) \\ 2 \mapsto (C, A) \\ 3 \mapsto (B, A) \\ 4 \mapsto (D, B) \\ 5 \mapsto (D, B) \\ 6 \mapsto (D, D) \end{cases}$$

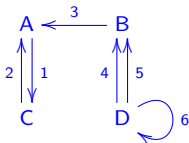
Nota: dígrafos não impõem restrições sobre os pares de nodos, permitindo laços e múltiplas arestas.

Dígrafos: matriz de incidência

Preenchimento da matriz de incidência de um dígrafo:

- uma aresta $A \rightarrow B$ registra 1 no nodo A e -1 no nodo B (nodos distintos)
- um laço $A \rightarrow A$ registra 2 no nodo A

Exemplo:



	A	B	C	D
1	1	0	-1	0
2	-1	0	1	0
3	-1	1	0	0
4	0	-1	0	1
5	0	-1	0	1
6	0	0	0	2

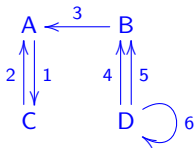
Nota:

- varia de autor para autor associar $(1/-1)$ ou $(-1/1)$ à origem/destino das setas. Nesta disciplinas vamos utilizar $(1/-1)$.
- é perdida a correlação entre a soma dos graus do grafo e a soma da matriz de incidência.

Dígrafos: matriz de adjacência

Preenchimento da matriz de adjacência de um dígrafo: a posição (A, B) registra a quantidade de arestas de A para B .

Exemplo:



	A	B	C	D
A	0	0	1	0
B	1	0	0	0
C	1	0	0	0
D	0	2	0	1

Nota: note que a matriz de adjacência não possui nenhuma restrição (exceto todos os números serem inteiros não-negativos).

Dígrafos: graus

Definição: cada vértice v em um dígrafo (V, E, conn) possui dois graus:

- grau de entrada $\text{deg}^-(v)$

$$\text{deg}^-(v) = |\{e \mid e \in E \wedge \text{tgt}(e) = v\}|$$

(número de arestas com destino v)

- grau de saída $\text{deg}^+(v)$

$$\text{deg}^+(v) = |\{e \mid e \in E \wedge \text{src}(e) = v\}|$$

(número de arestas com origem v)

Lema: em todo dígrafo $D = (V, E, \text{conn})$

$$\sum_{v \in V} \text{deg}^-(v) = \sum_{v \in V} \text{deg}^+(v) = |E|$$

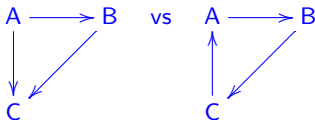
Dígrafos: passeios e ciclos

Noções relativas a passeios em dígrafos são as mesmas que em pseudografos, obviamente levando-se em consideração *direcionalidade*.

Classificação em dígrafo cíclico e acíclico também é mantida (existência ou não de ciclos).

Nota: a diferença entre um dígrafo cíclico e acíclico pode ser somente a direcionalidade das setas.

Exemplo:

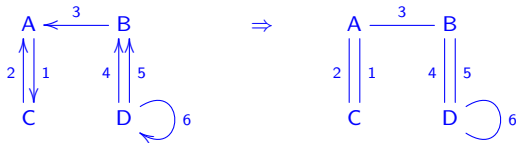


Grafos direcionados acíclicos (GDAs) são bastante utilizados para descrever um conjunto de tarefas e suas dependências (*workflow*).

Dígrafos: conectividade

Definição: o **grafo subjacente** de um dígrafo é um pseudografo com os mesmos vértices onde ignoramos a direcionalidade das arestas.

Exemplo:



Definição: um dígrafo é **fracamente conexo** quando seu *grafo subjacente* é conexo.

Definição: um dígrafo é **fortemente conexo** se entre qualquer par de nodos existe um caminho.

Conceitos relacionados à conectividade podem ser relacionados à conectividade forte ou fraca. Exemplo: componentes conexos.

Conteúdo

Grafos simples, incidência, adjacência

Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Grafos e dígrafos valorados

Existem muitos cenários onde é interessante anotar nodos ou arestas de um grafo (ou dígrafo) com valores:

Exemplo:

- definir custo (distância, velocidade, etc.) em arestas
- associar um custo ou recompensa a cada nodo
- determinar o tipo de uma determinada transição

Tipos de valores:

- discretos (alfabeto finito): rótulos ou etiquetas
- numéricos ($\mathbb{N}$, $\mathbb{R}$, ...): pesos

Associação é dada por *funções de valoração*.

Dígrafos valorados

Definição: um **dígrafo com pesos (arestas)** é uma tupla (V, E, conn, w) onde

- (V, E, conn) é um dígrafo
- $w : E \rightarrow \mathbb{R}$ é uma função que associa a cada aresta um número real.

Nota: pode-se definir dígrafos (e grafos) com pesos em nodos de forma similar, através de uma função $w_V : V \rightarrow \mathbb{R}$.

Dígrafos valorados: distância

Definição: o **peso de um passeio** em um grafo com pesos nas arestas é a soma dos pesos de todas as arestas do passeio.

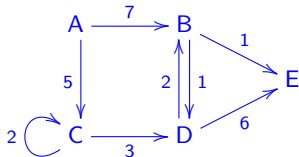
Definição: **distância** entre dois nós a e b , denotada $d(a, b)$:

$$d : V \times V \rightarrow \mathbb{N} \cup \{\infty, -\infty\}$$

$$d(a, b) = \begin{cases} 0 & \text{se } a = b \text{ e não houver ciclo negativo em a} \\ & \text{menor } \mathbf{peso} \text{ de passeio entre a e b} \\ -\infty & \text{se houver passeios entre a e b,} \\ & \text{mas não houver um menor peso} \\ \infty & \text{se a e b não estão conectados} \end{cases}$$

Dígrafos valorados: exemplo

Exemplo:



$V = \{A, B, C, D, E\}$

$E = \{a1, a2, a3, a4, a5, a6, a7, a8\}$

conn =

$a1 \mapsto (A, B)$

$a2 \mapsto (A, C)$

$a3 \mapsto (B, D)$

$a4 \mapsto (B, E)$

$a5 \mapsto (C, C)$

$a6 \mapsto (C, D)$

$a7 \mapsto (D, B)$

$a8 \mapsto (D, E)$

w =

$a1 \mapsto 7$

$a2 \mapsto 5$

$a3 \mapsto 1$

$a4 \mapsto 1$

$a5 \mapsto 2$

$a6 \mapsto 3$

$a7 \mapsto 2$

$a8 \mapsto 6$

Nota: é comum omitir os nomes das arestas na representação gráfica e apresentar somente os respectivos pesos.

Conteúdo

Grafos simples, incidência, adjacência
Isomorfismo, topologias comuns
Graus
Passeios
Conectividade
Operações sobre grafos
Grafos cíclicos, árvores e florestas
Grafos bipartidos e k-partidos
Pseudografos
Dígrafos
Grafos e dígrafos valorados

Busca em profundidade e busca em largura
Algoritmo de Dijkstra
Grafos eulerianos
Grafos hamiltonianos
Problema do caixeiro viajante
Árvores geradoras mínimas
Algoritmos de Prim e Kruskal
Grafos planares
Coloração de grafos
Enumeração de árvores
Coberturas de vértices
Emparelhamentos

Algoritmos de busca em grafos

Existem duas categorias básicas de algoritmos de busca em grafos:

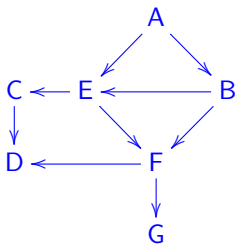
- busca em profundidade
- busca em largura (amplitude)

Algoritmos de busca podem ser utilizados para encontrar caminhos, detectar ciclos, obter componentes conexos de um grafo.

Veremos a seguir exemplos de algoritmos que usam tais estratégias (em Racket).

Exemplo: dígrafo acíclico

DAG1 =



Código Racket (lista de adjacências):

```
(define DAG1
  (list (list 'A 'B 'E)
        (list 'B 'E 'F)
        (list 'C 'D)
        (list 'D)
        (list 'E 'C 'F)
        (list 'F 'D 'G)
        (list 'G)))
```


Busca: profundidade vs largura

Exemplo: teste de conectividade entre nodos **a** e **b** em dígrafos.

Ideia: inicia-se no nodo **a**.

- se **a = b**, estão conectados.
- se não, tenta achar caminho para **b** a partir dos vizinhos de **a**.
- move-se pelo grafo até achar **b**, ou esgotarem-se as possibilidades de caminho.

Diferença das estratégias está na sequência de operações:

- em profundidade: se o primeiro vizinho (digamos **c**) do nodo atual não for o destino, testa-se os vizinhos de **c** antes de tentar os demais vizinhos do nodo atual.
- em largura: tenta inicialmente todos os vizinhos do nodo atual como destino, antes de começar a buscar pelos vizinhos dos vizinhos.

Função auxiliar: vizinhos

```
; vizinhos : símbolo, grafo -> lista-de-símbolos
; (vizinhos n G) retorna todos os nodos no grafo G
; que recebem arestas de n

(define (vizinhos n G)
  (cond
    ; erro, não achou nodo
    [(empty? G) empty]

    ; achou nodo, retorna nodos adjacentes
    [(equal? n (first (first G))) (rest (first G))]

    ; não é o nodo atual, continua busca no resto da lista
    [else (vizinhos n (rest G))]))
```

Conectividade (busca em largura)

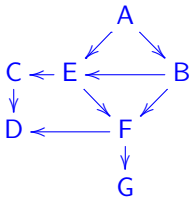
```
; conectados-larg? : simbolo simbolo grafo -> boolean
(define (conectados-larg? a b G)
  (busca-larg (list a) b G))

; busca-larg : lista-de-símbolos, simbolo, grafo -> boolean
(define (busca-larg la b G)
  (cond
    ; lista vazia?
    [(empty? la) false]
    ; destino pertence à lista de nodos atual?
    [(member b la) true]
    ; repete busca nos vizinhos dos vizinhos
    [else (busca-larg (todos-vizinhos la G) b G)]))

; todos-vizinhos: lista-de-simbolos, grafo -> lista-de-simbolos
(define (todos-vizinhos la G)
  (cond
    [(empty? la) empty]
    [else (append (vizinhos (first la) G)
                  (todos-vizinhos (rest la) G))]))
```

Conectividade (busca em largura): testes

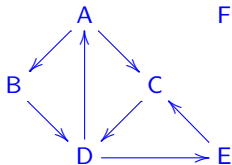
```
; testes
; A A:  '(A) => true
; A B:  '(A) => '(B E) => true
; B D:  '(B) => '(E F) => '(C F D G) => true
; G E:  '(G) => '() => false
; E A:  '(E) => '(C F) => '(D D G) => '() => false
```



Busca em dígrafos cíclicos

O algoritmo visto requer que o dígrafo seja acíclico, caso contrário há a possibilidade de se entrar em um laço de recursão infinita.

Exemplo de dígrafo cíclico:



```
(list  
  (list 'A 'B 'C)  
  (list 'B 'D)  
  (list 'C 'D)  
  (list 'D 'A 'E)  
  (list 'E 'C )  
  (list 'F)))
```

```
; A F: '(A) => '(B C) => '(D) => '(A E) => '(B C C) => ...
```

Busca em largura sobre dígrafos com ciclos: é preciso registrar os lugares já passados, e não retornar a eles. Usaremos uma lista p para tal.

Conectividade (busca em largura, detecta ciclos)

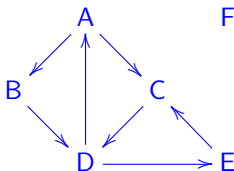
```
; conectados-larg? : símbolo símbolo grafo -> boolean
(define (conectados-larg? a b G)
  (busca-larg (list a) (list ) b G))

; busca-larg : lista-simb, simb, lista-simb, grafo -> boolean
(define (busca-larg la p b G)
  (cond
    ; lista vazia?
    [(empty? la) false]
    ; destino pertence à lista de nodos atual?
    [(member b la) true]
    ; repete busca nos vizinhos dos vizinhos
    [else (busca-larg (todos-vizinhos la p G) (append la p) b G)]))

; todos-vizinhos: lista-simb, lista-simb, grafo -> lista-simb
(define (todos-vizinhos la p G)
  (cond
    [(empty? la) empty]
    [else (append (filter (lambda (x) (not (member x p)))
                          (vizinhos (first la) G))
                  (todos-vizinhos (rest la) p G))]))
```

Conectividade (largura, ciclos): teste

```
; teste    l      p
; A F:    '(A)    '()      =>
;          '(B C)  '(A)      =>
;          '(D)    '(B C A)  =>
;          '(E)    '(D B C A) =>
;          '()     '(E D B C A) => false
```



Busca em largura: aplicações

Vários problemas de grafos podem ser resolvidos através de uma estratégia de busca em largura:

- conectividade entre nodos (visto)
- descoberta de caminho entre nodos
- cálculo de distância entre nodos
- cálculo de componentes conexos maximais

Vejamos agora uma implementação do problema de conectividade entre nodos, usando a estratégia de *busca em profundidade* (com detecção de ciclos).

Conectividade (profundidade,ciclos)

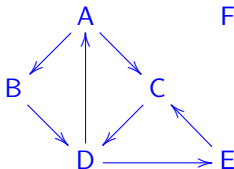
```
; conectados-prof? : simbolo simbolo grafo -> boolean
(define (conectados-prof? a b G)
  (busca-prof (list a) (list ) b G))

; busca-prof : lista-simb, simb, lista-simb, grafo -> bool
(define (busca-prof la p b G)
  (cond
    ; não há nodos na lista
    [(empty? la) false]
    ; o primeiro nodo é o destino?
    [(equal? (first la) b) true]
    ; busca em profundidade a partir do primeiro nodo.
    [else (busca-prof
            (append (novos-vizinhos (first la) p G) (rest la))
            (cons (first la) p) b G)]))

; novos-vizinhos: simb, lista-simb, grafo -> lista-simb
(define (novos-vizinhos a p G)
  (filter (lambda (x) (not (member x p)))
    (vizinhos a G)))
```

Conectividade (profundidade,ciclos): teste

```
; teste:  
;(conectados-prof? 'A 'F D2)  
; (busca-prof (list 'A)      empty                'F D2) =>  
; (busca-prof (list 'B 'C) (list 'A)              'F D2) =>  
; (busca-prof (list 'D 'C) (list 'B 'A)            'F D2) =>  
; (busca-prof (list 'E 'C) (list 'D 'B 'A)         'F D2) =>  
; (busca-prof (list 'C 'C) (list 'E 'D 'B 'A)      'F D2) =>  
; (busca-prof (list 'C)    (list 'C 'E 'D 'B 'A)   'F D2) =>  
; (busca-prof empty      (list 'C 'C 'E 'D 'B 'A) 'F D2) =>  
; false
```



Busca em profundidade: aplicações

Exemplos de problemas onde podemos utilizar busca em profundidade:

- encontrar caminhos entre nodos
- encontrar componentes conexos
- ordenamento topológico de nodos

Nota: busca em profundidade não é adequada para o cálculo de distâncias pois pode-se encontrar um caminho não-mínimo entre origem e destino.

Algoritmos de busca em grafos: exercícios

Exercício: Implemente algoritmos na linguagem Racket para resolver os seguintes problemas. Considere que os grafos são representados através de listas de adjacência (como visto nos slides). Quando for adequado, você pode construir as soluções modificando os algoritmos de busca apresentados.

Nos problemas abaixo, além do que é pedido, considere que o algoritmo recebe como entrada um dígrafo qualquer.

1. Dados dois vértices x e y , calcular um caminho qualquer entre eles.
2. Dados dois vértices x e y , calcular todos os caminhos possíveis entre eles.
3. Dados dois vértices x e y , calcular a distância entre eles.

Nos problemas abaixo, além do que é pedido, considere que o algoritmo recebe como entrada um *grafo simples*.

1. Dado um nodo x , calcular o componente conexo maximal que o contém.
2. Dado um conjunto de vértices, verificar se é um clique.
3. Dado um conjunto de vértices (sabidamente um clique), verificar se é maximal.

Conteúdo

Grafos simples, incidência, adjacência

Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Algoritmo de Dijkstra

Problema: calcular a distância de um dado nodo até qualquer outro nodo do grafo.

Algoritmo ingênuo: calcula-se todos os caminhos, e seus respectivos pesos totais, buscando o menor peso total.

Algoritmo proposto por Dijkstra (1959): explora o fato que caminhos mínimos só podem ser decompostos em caminhos ótimos.

Usado para determinar tanto a distância, quanto o respectivo caminho mínimo.

Veremos o algoritmo para o caso de grafo simples com pesos

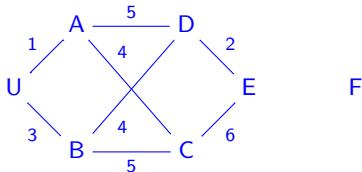
$$G = (V, E, w)$$

onde $w : 2^V \rightarrow \mathbb{R} \cup \{\infty\}$ associa um peso *não-negativo* a cada aresta.

Nota: vamos considerar $w(e) = \infty$ se $e \notin E$

Algoritmo de Dijkstra: exemplo

Exemplo:



- $d(U, U) = 0$
- $d(U, A) = 1$
- $d(U, B) = 3$
- $d(U, C) = 5$ (UAC)
- $d(U, D) = 6$ (UAD)
- $d(U, E) = 8$ (UADE)
- $d(U, F) = \infty$

Algoritmo de Dijkstra: pseudocódigo

Entrada: Um grafo com pesos não-negativos (V, E, w) , e um nodo $u \in V$

Saída : Uma tabela t associando cada nodo $v \in V$ à sua distância $d(u, v)$

begin

 // Inicialização da tabela t e conjunto de nodos S

for $v \in V$ **do**

if $v = u$ **then** $t(u) \leftarrow 0$;

else $t(v) \leftarrow w(\{u, v\})$;

$S \leftarrow \{u\}$;

 // Iteração

while $S \neq V$ e existe $z \notin S$ tal que $t(z) \neq \infty$ **do**

 Selecione $v \notin S$ tal que $t(v) = \min_{x \notin S} t(x)$;

$S \leftarrow S \cup \{v\}$;

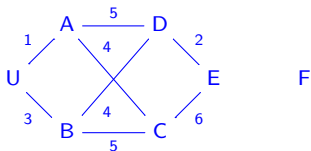
for $\{v, z\} \in E$ onde $z \notin S$ **do**

$t(v) \leftarrow \min\{t(z), t(v) + w(\{v, z\})\}$;

 // Finalização

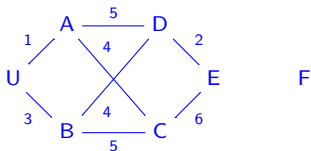
return t

Algoritmo de Dijkstra: execução



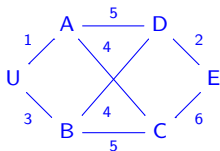
t_0	
U	0
A	1
B	3
C	∞
D	∞
E	∞
F	∞
S_0	
{U}	
min:	A

Algoritmo de Dijkstra: execução



t₀		t₁	
U	0	U	0
A	1	A	1
B	3	B	3
C	∞	C	5
D	∞	D	6
E	∞	E	∞
F	∞	F	∞
S₀		S₁	
{U}		{U, A}	
min: A		min: B	

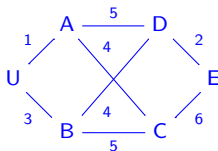
Algoritmo de Dijkstra: execução



F

t_0	t_1	t_2
U 0	U 0	U 0
A 1	A 1	A 1
B 3	B 3	B 3
C ∞	C 5	C 5
D ∞	D 6	D 6
E ∞	E ∞	E ∞
F ∞	F ∞	F ∞
S_0 {U}	S_1 {U, A}	S_2 {U, A, B}
min: A	min: B	min: C

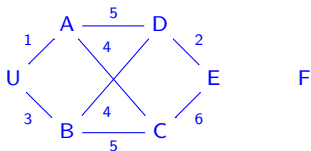
Algoritmo de Dijkstra: execução



F

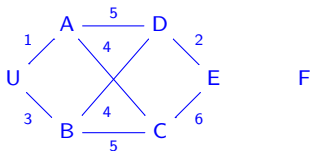
t_0	t_1	t_2	t_3
U 0	U 0	U 0	U 0
A 1	A 1	A 1	A 1
B 3	B 3	B 3	B 3
C ∞	C 5	C 5	C 5
D ∞	D 6	D 6	D 6
E ∞	E ∞	E ∞	E 11
F ∞	F ∞	F ∞	F ∞
S_0 {U}	S_1 {U, A}	S_2 {U, A, B}	S_3 {U, A, B, C}
min: A	min: B	min: C	min: D

Algoritmo de Dijkstra: execução



t₀	t₁	t₂	t₃	t₄
U 0	U 0	U 0	U 0	U 0
A 1	A 1	A 1	A 1	A 1
B 3	B 3	B 3	B 3	B 3
C ∞	C 5	C 5	C 5	C 5
D ∞	D 6	D 6	D 6	D 6
E ∞	E ∞	E ∞	E 11	E 8
F ∞	F ∞	F ∞	F ∞	F ∞
S₀ {U}	S₁ {U, A}	S₂ {U, A, B}	S₃ {U, A, B, C}	S₄ {U, A, B, C, D}
min: A	min: B	min: C	min: D	min: E

Algoritmo de Dijkstra: execução



t₀ U 0 A 1 B 3 C ∞ D ∞ E ∞ F ∞	t₁ U 0 A 1 B 3 C 5 D 6 E ∞ F ∞	t₂ U 0 A 1 B 3 C 5 D 6 E ∞ F ∞	t₃ U 0 A 1 B 3 C 5 D 6 E 11 F ∞	t₄ U 0 A 1 B 3 C 5 D 6 E 8 F ∞	t₅ U 0 A 1 B 3 C 5 D 6 E 8 F ∞
S₀ {U}	S₁ {U, A}	S₂ {U, A, B}	S₃ {U, A, B, C}	S₄ {U, A, B, C, D}	S₅ {U, A, B, C, D, E}
min: A	min: B	min: C	min: D	min: E	para!

Algoritmo de Dijkstra: prova de correção

Teorema: dado um (dí)grafo G com pesos não-negativos nas arestas e um nodo $u \in V(G)$, o algoritmo de Dijkstra computa $d(u, z)$ para todo $z \in V(G)$.

Demonstração: será provado que, a cada iteração

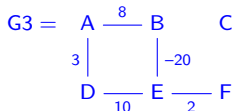
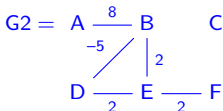
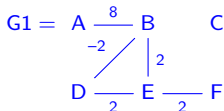
1. para $z \in S$, $t(z) = d(u, z)$.
2. para $z \notin S$, $t(z)$ é a menor distância de um caminho $c(u, z)$ alcançando z em exatamente um passo a partir de algum nodo em S .

por indução no tamanho do conjunto S .

Algoritmo de Dijkstra: limitações

Uma restrição importante ao algoritmo de Dijkstra é o fato de que os grafos necessariamente precisam possuir pesos não-negativos.

Exemplo: considere os seguintes grafos com pesos arbitrários:



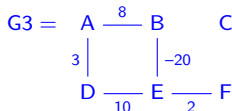
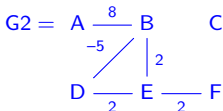
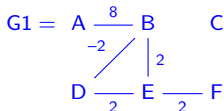
Responda:

- Qual a distância entre A e F?
- O que o algoritmo de Dijkstra apontaria como menor distância se fosse aplicado?
- Por que o algoritmo de Dijkstra não funciona em alguns desses grafos?

Algoritmo de Dijkstra: limitações

Uma restrição importante ao algoritmo de Dijkstra é o fato de que os grafos necessariamente precisam possuir pesos não-negativos.

Exemplo: considere os seguintes grafos com pesos arbitrários:



Responda:

- Qual a distância entre A e F?
- O que o algoritmo de Dijkstra apontaria como menor distância se fosse aplicado?
- Por que o algoritmo de Dijkstra não funciona em alguns desses grafos?

Exercício: pesquise sobre os algoritmos de menor caminho conhecidos como Bellman-Ford e Johnson.

Conteúdo

Grafos simples, incidência, adjacência
Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

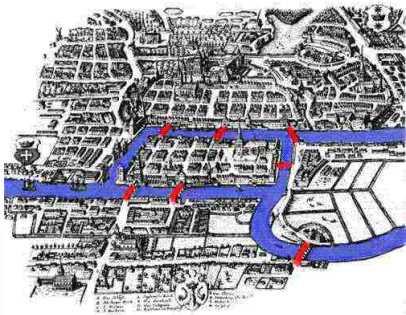
Coberturas de vértices

Emparelhamentos

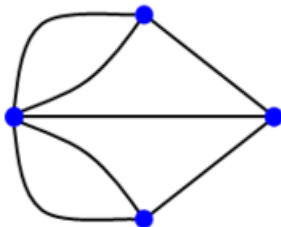
Problema das pontes de Königsberg

Considerado o primeiro problema de teoria dos grafos, resolvido em 1736 por Leonard Euler.

Problema: dada a disposição das pontes da cidade, há algum caminho que passe por cada uma delas uma única vez?



Representação das pontes de Königsberg



Multigrafo equivalente

Circuitos e grafos eulerianos

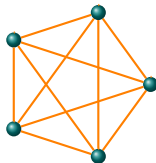
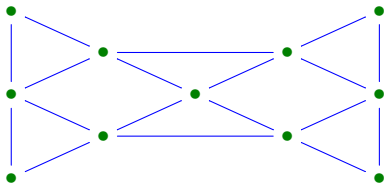
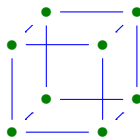
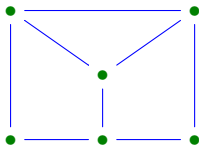
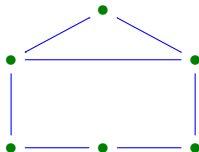
Definição: uma trilha/circuito é **euleriana(o)** sss passa por todas as arestas do grafo.

Definição: um **grafo** é **semi-euleriano** sss possui uma trilha euleriana ou é nulo (sem arestas).

Definição: um **grafo** é **euleriano** sss possui um circuito euleriano ou é nulo (sem arestas).

Circuitos e grafos eulerianos: exercício

Exercício: classifique os seguintes grafos de acordo com a existência ou não de trilhas/circuitos eulerianos:



Caracterização de grafos eulerianos

Lema: dado um *grafo simples* finito G , se $\deg(v) \geq 2$ para todo $v \in V$, então G é cíclico.

Demonstração: considere um caminho c em G que seja aberto (pontos extremos a e b são distintos) e maximal (não está contido em nenhum outro caminho aberto).

$$c(a, b) = a \text{ --- } \bullet \text{ --- } \bullet \text{ --- } \dots \text{ --- } b$$

Como G é finito, caminhos maximais existem em cada componente conexo.

Como o grau de todo nó é maior que 2, a deve estar conectados a no mínimo mais um nó x .

Se $x \notin c$, teríamos uma caminho maior $c'(x, b)$, e portanto haveria uma contradição com o fato de c ser maximal.

Portanto, a deve estar conectado a algum $x \in c$, formando um ciclo.

$$a \text{ --- } \bullet \text{ --- } x \text{ --- } \dots \text{ --- } b$$


Caracterização de grafos eulerianos (cont.)

Definição: um componente conexo é não-trivial sss possui ao menos uma aresta.

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo *não-trivial* e os graus de todos os nós são pares.

Demonstração: ($\Rightarrow$). G possui circuito euleriano, que chamaremos c .



Caracterização de grafos eulerianos (cont.)

Definição: um componente conexo é não-trivial sss possui ao menos uma aresta.

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo *não-trivial* e os graus de todos os nós são pares.

Demonstração: ($\Rightarrow$). G possui circuito euleriano, que chamaremos c .



Um dado nodo n pode ocorrer ou não em c :

- $n \notin c$ caminho (não há arestas incidentes): grau 0 (par).

Caracterização de grafos eulerianos (cont.)

Definição: um componente conexo é não-trivial sss possui ao menos uma aresta.

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo *não-trivial* e os graus de todos os nós são pares.

Demonstração: ($\Rightarrow$). G possui circuito euleriano, que chamaremos c .



Um dado nodo n pode ocorrer ou não em c :

- $n \notin c$ caminho (não há arestas incidentes): grau 0 (par).
- $n \in c$: cada ocorrência do nodo contará um acréscimo de 2 no grau, visto que está incidente a duas arestas distintas (c é euleriano).

Caracterização de grafos eulerianos (cont.)

Definição: um componente conexo é não-trivial sss possui ao menos uma aresta.

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo *não-trivial* e os graus de todos os nós são pares.

Demonstração: ($\Rightarrow$). G possui circuito euleriano, que chamaremos c .



Um dado nodo n pode ocorrer ou não em c :

- $n \notin c$ caminho (não há arestas incidentes): grau 0 (par).
- $n \in c$: cada ocorrência do nodo contará um acréscimo de 2 no grau, visto que está incidente a duas arestas distintas (c é euleriano). Como o circuito contém todas as arestas, não há outras contribuições para o grau de cada nodo.

Caracterização de grafos eulerianos (cont.)

Definição: um componente conexo é não-trivial sss possui ao menos uma aresta.

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo *não-trivial* e os graus de todos os nós são pares.

Demonstração: ($\Rightarrow$). G possui circuito euleriano, que chamaremos c .



Um dado nodo n pode ocorrer ou não em c :

- $n \notin c$ caminho (não há arestas incidentes): grau 0 (par).
- $n \in c$: cada ocorrência do nodo contará um acréscimo de 2 no grau, visto que está incidente a duas arestas distintas (c é euleriano). Como o circuito contém todas as arestas, não há outras contribuições para o grau de cada nodo. Logo, o grau de n é par, sendo 2 vezes o número de ocorrências do nó em c .

Caracterização de grafos eulerianos (cont.)

Definição: um componente conexo é não-trivial sss possui ao menos uma aresta.

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo *não-trivial* e os graus de todos os nós são pares.

Demonstração: ($\Rightarrow$). G possui circuito euleriano, que chamaremos c .



Um dado nodo n pode ocorrer ou não em c :

- $n \notin c$ caminho (não há arestas incidentes): grau 0 (par).
- $n \in c$: cada ocorrência do nodo contará um acréscimo de 2 no grau, visto que está incidente a duas arestas distintas (c é euleriano). Como o circuito contém todas as arestas, não há outras contribuições para o grau de cada nodo. Logo, o grau de n é par, sendo 2 vezes o número de ocorrências do nó em c .

Assumir que há mais de um componente conexo não-trivial (com arestas) gera contradição com o fato de haver um circuito euleriano, visto que não seria possível haver caminho entre dois componentes conexos distintos.

Caracterização de grafos eulerianos (cont.)

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo *não-trivial* e os graus de todos os nós são pares.

Caracterização de grafos eulerianos (cont.)

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo *não-trivial* e os graus de todos os nós são pares.

Demonstração: $(\Leftarrow)$ por indução no tamanho de E .

- **caso base:** $|E| = 0$. O grafo é euleriano por definição.

Caracterização de grafos eulerianos (cont.)

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo não-trivial e os graus de todos os nós são pares.

Demonstração: ($\Leftarrow$) por indução no tamanho de E .

- **caso base:** $|E| = 0$. O grafo é euleriano por definição.
- **passo indutivo:** como há nodos de grau maior que zero, existe ao menos um componente conexo não-trivial.

Caracterização de grafos eulerianos (cont.)

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo não-trivial e os graus de todos os nós são pares.

Demonstração: ($\Leftarrow$) por indução no tamanho de E .

- **caso base:** $|E| = 0$. O grafo é euleriano por definição.
- **passo indutivo:** como há nodos de grau maior que zero, existe ao menos um componente conexo não-trivial. Pelo lema anterior, sabemos que existe um subgrafo ciclo $C = (V_C, E_C)$ em G .

Caracterização de grafos eulerianos (cont.)

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo não-trivial e os graus de todos os nós são pares.

Demonstração: ($\Leftarrow$) por indução no tamanho de E .

- **caso base:** $|E| = 0$. O grafo é euleriano por definição.
- **passo indutivo:** como há nodos de grau maior que zero, existe ao menos um componente conexo não-trivial. Pelo lema anterior, sabemos que existe um subgrafo ciclo $C = (V_C, E_C)$ em G . O grau de todos os nodos em C é 2.

Caracterização de grafos eulerianos (cont.)

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo não-trivial e os graus de todos os nós são pares.

Demonstração: ($\Leftarrow$) por indução no tamanho de E .

- **caso base:** $|E| = 0$. O grafo é euleriano por definição.
- **passo indutivo:** como há nodos de grau maior que zero, existe ao menos um componente conexo não-trivial. Pelo lema anterior, sabemos que existe um subgrafo ciclo $C = (V_C, E_C)$ em G . O grau de todos os nodos em C é 2. O grafo $G' = G - E_C$ possui menos arestas que G , todos os graus de nodos são pares, e cada componente conexo não-trivial de G' possui um vértice em comum com C em G .

Caracterização de grafos eulerianos (cont.)

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo não-trivial e os graus de todos os nós são pares.

Demonstração: ($\Leftarrow$) por indução no tamanho de E .

- **caso base:** $|E| = 0$. O grafo é euleriano por definição.
- **passo indutivo:** como há nodos de grau maior que zero, existe ao menos um componente conexo não-trivial. Pelo lema anterior, sabemos que existe um subgrafo ciclo $C = (V_C, E_C)$ em G . O grau de todos os nodos em C é 2. O grafo $G' = G - E_C$ possui menos arestas que G , todos os graus de nodos são pares, e cada componente conexo não-trivial de G' possui um vértice em comum com C em G .

Hipótese indutiva: como $|G'| < |G|$, cada componente conexo de G' ou é nulo, ou possui circuito euleriano.

Caracterização de grafos eulerianos (cont.)

Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo não-trivial e os graus de todos os nós são pares.

Demonstração: ($\Leftarrow$) por indução no tamanho de E .

- **caso base:** $|E| = 0$. O grafo é euleriano por definição.
- **passo indutivo:** como há nodos de grau maior que zero, existe ao menos um componente conexo não-trivial. Pelo lema anterior, sabemos que existe um subgrafo ciclo $C = (V_C, E_C)$ em G . O grau de todos os nodos em C é 2. O grafo $G' = G - E_C$ possui menos arestas que G , todos os graus de nodos são pares, e cada componente conexo não-trivial de G' possui um vértice em comum com C em G .

Hipótese indutiva: como $|G'| < |G|$, cada componente conexo de G' ou é nulo, ou possui circuito euleriano. *Resultado:* Para construir o caminho euleriano final, percorre-se C entrando nos circuitos eulerianos dos componentes conexos de G' ao passar por um nodo comum de C e G' .

Caracterização de grafos eulerianos (cont.)

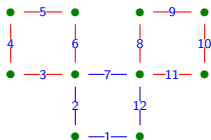
Teorema: um *grafo simples* finito $G = (V, E)$ é euleriano sss possui no máximo um componente conexo não-trivial e os graus de todos os nós são pares.

Demonstração: ($\Leftarrow$) por indução no tamanho de E .

- **caso base:** $|E| = 0$. O grafo é euleriano por definição.
- **passo indutivo:** como há nodos de grau maior que zero, existe ao menos um componente conexo não-trivial. Pelo lema anterior, sabemos que existe um subgrafo ciclo $C = (V_C, E_C)$ em G . O grau de todos os nodos em C é 2. O grafo $G' = G - E_C$ possui menos arestas que G , todos os graus de nodos são pares, e cada componente conexo não-trivial de G' possui um vértice em comum com C em G .

Hipótese indutiva: como $|G'| < |G|$, cada componente conexo de G' ou é nulo, ou possui circuito euleriano. *Resultado:* Para construir o caminho euleriano final, percorre-se C entrando nos circuitos eulerianos dos componentes conexos de G' ao passar por um nodo comum de C e G' .

Exemplo:



Corolário: todo grafo euleriano pode ser decomposto em ciclos disjuntos nas arestas (podem compartilhar nodos).

Conteúdo

Grafos simples, incidência, adjacência

Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

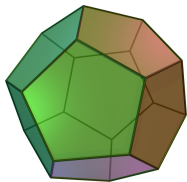
Coberturas de vértices

Emparelhamentos

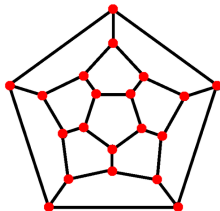
Problema do ciclo hamiltoniano

Problema: achar um caminho fechado nos vértices de um dodecaedro (poliedro regular de 12 faces) passando por cada vértices (ponta) uma única vez.

Problema proposto por Hamilton em 1857. Foi produzido um jogo de tabuleiro baseado neste problema (Icosian game).



Dodecaedro



Visão planar do dodecaedro



Jogo "Icosian game"

Imagens: <http://puzzlemuseum.com/month/picm02/200207icosian.htm>,
<http://en.wikipedia.org/wiki/Dodecahedron>

Ciclos e grafos hamiltonianos

Podemos reproduzir o problema de achar caminhos que passam uma e somente uma vez em cada vértices sobre um grafo qualquer.

Definição: um caminho/ciclo é **hamiltoniano** sss passa por todos os vértices do grafo.

Ciclos e grafos hamiltonianos

Podemos reproduzir o problema de achar caminhos que passam uma e somente uma vez em cada vértices sobre um grafo qualquer.

Definição: um caminho/ciclo é **hamiltoniano** sss passa por todos os vértices do grafo.

Definição: um **grafo** é **semi-hamiltoniano** sss possui um caminho hamiltoniano ou possui 1 vértice.

Ciclos e grafos hamiltonianos

Podemos reproduzir o problema de achar caminhos que passam uma e somente uma vez em cada vértices sobre um grafo qualquer.

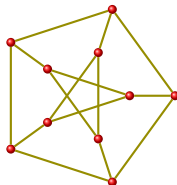
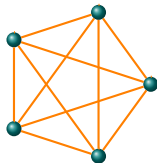
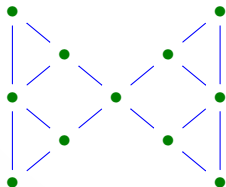
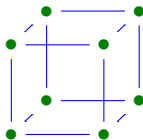
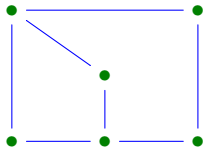
Definição: um caminho/ciclo é **hamiltoniano** sss passa por todos os vértices do grafo.

Definição: um **grafo** é **semi-hamiltoniano** sss possui um caminho hamiltoniano ou possui 1 vértice.

Definição: um **grafo** é **hamiltoniano** sss possui um ciclo hamiltoniano ou possui 1 vértice.

Ciclos e grafos hamiltonianos: exercício

Determine a classificação dos seguintes grafos de acordo com a existência ou não de ciclos/caminhos hamiltonianos.



Caracterização de grafos hamiltonianos

Grafos eulerianos possuem uma caracterização precisa em termos de propriedades de grau dos vértices. Podemos dizer se um grafo é euleriano sem precisar construir o circuito euleriano correspondente.

Caracterização de grafos hamiltonianos

Grafos eulerianos possuem uma caracterização precisa em termos de propriedades de grau dos vértices. Podemos dizer se um grafo é euleriano sem precisar construir o circuito euleriano correspondente.

No caso de grafos hamiltonianos, não se sabe até o momento um conjunto de condições simultaneamente suficientes e necessárias que os caracterizem completamente.

Caracterização de grafos hamiltonianos

Grafos eulerianos possuem uma caracterização precisa em termos de propriedades de grau dos vértices. Podemos dizer se um grafo é euleriano sem precisar construir o circuito euleriano correspondente.

No caso de grafos hamiltonianos, não se sabe até o momento um conjunto de condições simultaneamente suficientes e necessárias que os caracterizem completamente.

Exemplo: condições necessárias para um grafo simples G ser hamiltoniano:

1. $\delta(G) \geq 2$
2. G é conexo

Notem que elas não são suficientes, como mostra o grafo do canto inferior esquerdo do slide anterior.

Condição suficiente para grafos hamiltonianos

Ideia: se o grafo contém arestas suficientes, ele certamente é hamiltoniano.

Exemplo: grafos completos $K_3, K_4, K_5, \dots$

Condição suficiente para grafos hamiltonianos

Ideia: se o grafo contém arestas suficientes, ele certamente é hamiltoniano.

Exemplo: grafos completos $K_3, K_4, K_5, \dots$

À medida que aumenta o valor $\delta(G)$ (grau mínimo do grafo), existem mais interconexões entre nodos arbitrários.

Pergunta: qual o menor $\delta(G)$ de um grafo para que tenhamos certeza que ele é hamiltoniano?

Teorema de Dirac

Teorema: (Dirac, 1952) se $G = (V, E)$ é um grafo simples finito onde $|V| \geq 3$ e $\delta(G) \geq |V|/2$, então G é hamiltoniano.

Teorema de Dirac

Teorema: (Dirac, 1952) se $G = (V, E)$ é um grafo simples finito onde $|V| \geq 3$ e $\delta(G) \geq |V|/2$, então G é hamiltoniano.

Demonstração: baseada em grafos não-hamiltonianos maximais e contradição.

Teorema de Dirac

Teorema: (Dirac, 1952) se $G = (V, E)$ é um grafo simples finito onde $|V| \geq 3$ e $\delta(G) \geq |V|/2$, então G é hamiltoniano.

Demonstração: baseada em grafos não-hamiltonianos maximais e contradição.

Definição: um grafo $G = (V, E)$ é **não-hamiltoniano maximal** sss G não é hamiltoniano e, para qualquer $e \notin E$, temos que $(V, E \cup \{e\})$ é hamiltoniano.

Teorema de Dirac

Teorema: (Dirac, 1952) se $G = (V, E)$ é um grafo simples finito onde $|V| \geq 3$ e $\delta(G) \geq |V|/2$, então G é hamiltoniano.

Demonstração: baseada em grafos não-hamiltonianos maximais e contradição.

Definição: um grafo $G = (V, E)$ é **não-hamiltoniano maximal** sss G não é hamiltoniano e, para qualquer $e \notin E$, temos que $(V, E \cup \{e\})$ é hamiltoniano.

Intuição: um grafo não-hamiltoniano maximal possui o maior número possível de arestas sem ser hamiltoniano. Se colocarmos qualquer aresta a mais, então ele se torna hamiltoniano. Note que ele deve ser certamente semi-hamiltoniano (para poder fechar o ciclo com a aresta extra).

Teorema de Dirac

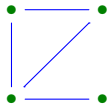
Teorema: (Dirac, 1952) se $G = (V, E)$ é um grafo simples finito onde $|V| \geq 3$ e $\delta(G) \geq |V|/2$, então G é hamiltoniano.

Demonstração: baseada em grafos não-hamiltonianos maximais e contradição.

Definição: um grafo $G = (V, E)$ é **não-hamiltoniano maximal** sss G não é hamiltoniano e, para qualquer $e \notin E$, temos que $(V, E \cup \{e\})$ é hamiltoniano.

Intuição: um grafo não-hamiltoniano maximal possui o maior número possível de arestas sem ser hamiltoniano. Se colocarmos qualquer aresta a mais, então ele se torna hamiltoniano. Note que ele deve ser certamente semi-hamiltoniano (para poder fechar o ciclo com a aresta extra).

Exemplo: com quatro vértices, temos o seguinte grafo não-hamiltoniano maximal (a menos de isomorfismo):



Teorema de Dirac (2)

Vamos mostrar que não existe grafo não-hamiltoniano maximal $G = (V, E)$ com $\delta(G) = n/2$, onde $n = |V|$. Para isso, assumimos que existe, e vamos obter uma contradição.

Teorema de Dirac (2)

Vamos mostrar que não existe grafo não-hamiltoniano maximal $G = (V, E)$ com $\delta(G) = n/2$, onde $n = |V|$. Para isso, assumimos que existe, e vamos obter uma contradição.

Do grafo não-hamiltoniano maximal G , tomamos um caminho hamiltoniano aberto c (e portanto, possuindo n vértices).

$$c = a \text{ --- } \dots \text{ --- } \bullet \text{ --- } \bullet \text{ --- } \dots \text{ --- } b$$

Teorema de Dirac (2)

Vamos mostrar que não existe grafo não-hamiltoniano maximal $G = (V, E)$ com $\delta(G) = n/2$, onde $n = |V|$. Para isso, assumimos que existe, e vamos obter uma contradição.

Do grafo não-hamiltoniano maximal G , tomamos um caminho hamiltoniano aberto c (e portanto, possuindo n vértices).

$$c = a \text{ --- } \dots \text{ --- } \bullet \text{ --- } \bullet \text{ --- } \dots \text{ --- } b$$

Se algum vizinho de a sucede um vizinho de b no caminho c , certamente há um ciclo (desviando da aresta central abaixo), e o grafo é hamiltoniano (e, portanto, chegamos a uma contradição com o fato de G ser não-hamiltoniano).

Teorema de Dirac (2)

Vamos mostrar que não existe grafo não-hamiltoniano maximal $G = (V, E)$ com $\delta(G) = n/2$, onde $n = |V|$. Para isso, assumimos que existe, e vamos obter uma contradição.

Do grafo não-hamiltoniano maximal G , tomamos um caminho hamiltoniano aberto c (e portanto, possuindo n vértices).

$$c = a \text{ --- } \dots \text{ --- } \bullet \text{ --- } \bullet \text{ --- } \dots \text{ --- } b$$

Se algum vizinho de a sucede um vizinho de b no caminho c , certamente há um ciclo (desviando da aresta central abaixo), e o grafo é hamiltoniano (e, portanto, chegamos a uma contradição com o fato de G ser não-hamiltoniano).

$$c = a \text{ --- } \dots \text{ --- } \bullet \text{ --- } \bullet \text{ --- } \dots \text{ --- } b$$

Teorema de Dirac (2)

Vamos mostrar que não existe grafo não-hamiltoniano maximal $G = (V, E)$ com $\delta(G) = n/2$, onde $n = |V|$. Para isso, assumimos que existe, e vamos obter uma contradição.

Do grafo não-hamiltoniano maximal G , tomamos um caminho hamiltoniano aberto c (e portanto, possuindo n vértices).

$$c = a \text{ --- } \dots \text{ --- } \bullet \text{ --- } \bullet \text{ --- } \dots \text{ --- } b$$

Se algum vizinho de a sucede um vizinho de b no caminho c , certamente há um ciclo (desviando da aresta central abaixo), e o grafo é hamiltoniano (e, portanto, chegamos a uma contradição com o fato de G ser não-hamiltoniano).

$$c = a \text{ --- } \dots \text{ --- } \bullet \text{ --- } \bullet \text{ --- } \dots \text{ --- } b$$

Basta mostrar que com $\delta(G) = n/2$ a situação acima sempre ocorrerá.

Teorema de Dirac (3)

Vamos chamar de V_a o conjunto de posições de **antecessores** de vizinhos de a (de 1 a $n-1$), e de V_b o conjunto de posições dos vizinhos de b (de 1 a $n-1$).

Teorema de Dirac (3)

Vamos chamar de V_a o conjunto de posições de **antecessores** de vizinhos de a (de 1 a $n-1$), e de V_b o conjunto de posições dos vizinhos de b (de 1 a $n-1$).

Suponha $\delta(G) = n/2$. Portanto tanto a quanto b possuem $n/2$ vizinhos, todos no caminho c (pois c é hamiltoniano). Consequentemente, $|V_a| = |V_b| = n/2$.

Teorema de Dirac (3)

Vamos chamar de V_a o conjunto de posições de **antecessores** de vizinhos de a (de 1 a $n-1$), e de V_b o conjunto de posições dos vizinhos de b (de 1 a $n-1$).

Suponha $\delta(G) = n/2$. Portanto tanto a quanto b possuem $n/2$ vizinhos, todos no caminho c (pois c é hamiltoniano). Consequentemente, $|V_a| = |V_b| = n/2$.

Agora mostramos que $V_a \cap V_b$ não é vazio, somando a cardinalidade dos dois conjuntos.

$$n = |V_a| + |V_b| = |V_a \cup V_b| + |V_a \cap V_b|$$

Teorema de Dirac (3)

Vamos chamar de V_a o conjunto de posições de **antecessores** de vizinhos de a (de 1 a $n-1$), e de V_b o conjunto de posições dos vizinhos de b (de 1 a $n-1$).

Suponha $\delta(G) = n/2$. Portanto tanto a quanto b possuem $n/2$ vizinhos, todos no caminho c (pois c é hamiltoniano). Consequentemente, $|V_a| = |V_b| = n/2$.

Agora mostramos que $V_a \cap V_b$ não é vazio, somando a cardinalidade dos dois conjuntos.

$$n = |V_a| + |V_b| = |V_a \cup V_b| + |V_a \cap V_b|$$

Como o índice n não ocorre nem em V_a nem em V_b , certamente $|V_a \cup V_b| < n$. Portanto, $|V_a \cap V_b| \geq 1$.

Teorema de Dirac (3)

Vamos chamar de V_a o conjunto de posições de **antecessores** de vizinhos de a (de 1 a $n-1$), e de V_b o conjunto de posições dos vizinhos de b (de 1 a $n-1$).

Suponha $\delta(G) = n/2$. Portanto tanto a quanto b possuem $n/2$ vizinhos, todos no caminho c (pois c é hamiltoniano). Consequentemente, $|V_a| = |V_b| = n/2$.

Agora mostramos que $V_a \cap V_b$ não é vazio, somando a cardinalidade dos dois conjuntos.

$$n = |V_a| + |V_b| = |V_a \cup V_b| + |V_a \cap V_b|$$

Como o índice n não ocorre nem em V_a nem em V_b , certamente $|V_a \cup V_b| < n$. Portanto, $|V_a \cap V_b| \geq 1$.

Como há no mínimo um índice comum entre antecessores de vizinhos de a e vizinhos de b , a situação descrita no slide anterior ocorre, havendo um ciclo hamiltoniano em G (o que é uma contradição com o fato de ser não-hamiltoniano).

Teorema de Dirac (3)

Vamos chamar de V_a o conjunto de posições de **antecessores** de vizinhos de a (de 1 a $n-1$), e de V_b o conjunto de posições dos vizinhos de b (de 1 a $n-1$).

Suponha $\delta(G) = n/2$. Portanto tanto a quanto b possuem $n/2$ vizinhos, todos no caminho c (pois c é hamiltoniano). Consequentemente, $|V_a| = |V_b| = n/2$.

Agora mostramos que $V_a \cap V_b$ não é vazio, somando a cardinalidade dos dois conjuntos.

$$n = |V_a| + |V_b| = |V_a \cup V_b| + |V_a \cap V_b|$$

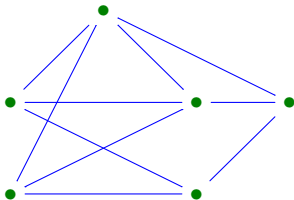
Como o índice n não ocorre nem em V_a nem em V_b , certamente $|V_a \cup V_b| < n$. Portanto, $|V_a \cap V_b| \geq 1$.

Como há no mínimo um índice comum entre antecessores de vizinhos de a e vizinhos de b , a situação descrita no slide anterior ocorre, havendo um ciclo hamiltoniano em G (o que é uma contradição com o fato de ser não-hamiltoniano).

Portanto, não é possível haver um grafo maximal não-hamiltoniano com $\delta(G) = |V|/2$. O argumento vale para qualquer $\delta(G)$ maior que $n/2$. □

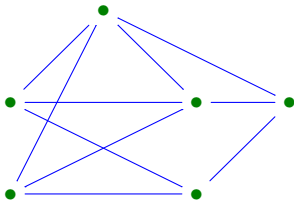
Teorema de Dirac (4)

Exemplo: o grafo abaixo tem 6 nodos e grau mínimo 3. Portanto, pelo teorema de Dirac, é hamiltoniano.



Teorema de Dirac (4)

Exemplo: o grafo abaixo tem 6 nodos e grau mínimo 3. Portanto, pelo teorema de Dirac, é hamiltoniano.



Existem outros teoremas que garantem que certos tipos de grafos são hamiltonianos. Contudo, nenhum deles engloba todos os possíveis grafos hamiltonianos.

Conteúdo

Grafos simples, incidência, adjacência
Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

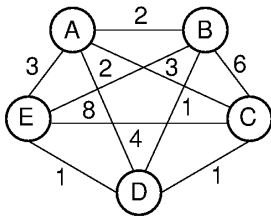
Emparelhamentos

Problema do caixeiro viajante

Intuição: imagine que um caixeiro viajante parta de uma cidade e queira vender seus produtos em todas as demais cidades de um determinado estado, voltando para a cidade de origem. Ele não quer passar mais de uma vez por cada cidade e, além disso, ele gostaria de andar o mínimo possível. A resposta do problema é a rota que o caixeiro viajante deve utilizar para alcançar seus objetivos.

Problema: dado um grafo com pesos nas arestas (não-negativos), determinar um ciclo hamiltoniano de peso mínimo.

Exercício: apresente um ciclo hamiltoniano de menor peso para o seguinte grafo:



Problema do caixeiro viajante

Para um grafo completo K_n , há $n!/(2 * n)$ ciclos hamiltonianos distintos.

Problema do caixeiro viajante

Para um grafo completo K_n , há $n!/(2 * n)$ ciclos hamiltonianos distintos.

No exemplo anterior, temos que averiguar o peso de 12 ciclos distintos.

Problema do caixeiro viajante

Para um grafo completo K_n , há $n!/(2 * n)$ ciclos hamiltonianos distintos.

No exemplo anterior, temos que averiguar o peso de 12 ciclos distintos.

Para um grafo completo com 50 vértices, número de caminhos distintos:
304140932017133780436126081660647688443776415689605120000000000

Problema do caixeiro viajante

Para um grafo completo K_n , há $n!/(2 * n)$ ciclos hamiltonianos distintos.

No exemplo anterior, temos que averiguar o peso de 12 ciclos distintos.

Para um grafo completo com 50 vértices, número de caminhos distintos:

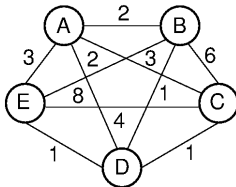
304140932017133780436126081660647688443776415689605120000000000

Sobre complexidade:

- problema de decisão (há ciclo hamiltoniano de peso menor que x ?): NP-completo
- problema de otimização (encontre o ciclo hamiltoniano de menor peso): NP-difícil
- por tais razões, é comum o emprego de *heurísticas* eficientes para se chegar a resultados próximos à solução, porém sem garantias sobre tê-la alcançado.

Problema do caixeiro viajante

Exemplo:



Ciclos e respectivos pesos (calculados por força bruta):

("abedc", 9)	("acdbe", 10)
("abdce", 11)	("abdec", 11)
("abcde", 13)	("acbde", 14)
("abecd", 17)	("acebd", 18)
("acbed", 20)	("adcbe", 20)
("abced", 21)	("adbce", 22)

Conteúdo

Grafos simples, incidência, adjacência
Isomorfismo, topologias comuns
Graus
Passeios
Conectividade
Operações sobre grafos
Grafos cíclicos, árvores e florestas
Grafos bipartidos e k-partidos
Pseudografos
Dígrafos
Grafos e dígrafos valorados

Busca em profundidade e busca em largura
Algoritmo de Dijkstra
Grafos eulerianos
Grafos hamiltonianos
Problema do caixeiro viajante
Árvores geradoras mínimas
Algoritmos de Prim e Kruskal
Grafos planares
Coloração de grafos
Enumeração de árvores
Coberturas de vértices
Emparelhamentos

Subgrafos e árvores geradoras

Definição: Seja $G = (V, E)$ um grafo simples. Um **subgrafo gerador** de G é um grafo $G' = (V', E')$ onde $G' \subseteq G$ e $V' = V$.

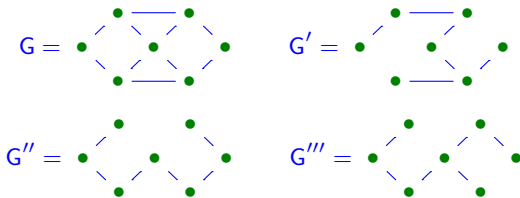
Definição: Um subgrafo gerador de G que é uma árvore é chamado uma **árvore geradora** de G .

Subgrafos e árvores geradoras

Definição: Seja $G = (V, E)$ um grafo simples. Um **subgrafo gerador** de G é um grafo $G' = (V', E')$ onde $G' \subseteq G$ e $V' = V$.

Definição: Um subgrafo gerador de G que é uma árvore é chamado uma **árvore geradora** de G .

Exemplo: G' , G'' e G''' são árvores geradoras distintas de G



Nota: árvores geradoras também são conhecidas como árvores de extensão, árvores de cobertura, árvores espalhadas ou árvores de espalhamento. Termo em inglês: *spanning tree*.

Árvores geradoras mínimas

A definição de um árvore geradora para um grafo com pesos nas arestas é a mesma que a definição na versão sem pesos.

Árvores geradoras mínimas

A definição de um árvore geradora para um grafo com pesos nas arestas é a mesma que a definição na versão sem pesos.

Definição: o **peso** de uma árvore geradora $A = (V, E, w)$ de G (grafo com pesos) é a soma

$$W(A) = \sum_{e \in E} w(e)$$

Árvores geradoras mínimas

A definição de um árvore geradora para um grafo com pesos nas arestas é a mesma que a definição na versão sem pesos.

Definição: o **peso** de uma árvore geradora $A = (V, E, w)$ de G (grafo com pesos) é a soma

$$W(A) = \sum_{e \in E} w(e)$$

Definição: dado um grafo com pesos G , uma **árvore geradora mínima** (AGM) é uma árvore geradora com peso menor ou igual a todas as demais árvores geradoras de G (isto é, uma árvore que minimiza W)

Descoberta de árvores geradoras mínimas

Problema: dado um grafo simples com pesos nas arestas G , determinar uma árvore geradora mínima para ele.

Descoberta de árvores geradoras mínimas

Problema: dado um grafo simples com pesos nas arestas G , determinar uma árvore geradora mínima para ele.

Aplicações: conectar estações situadas em diferentes localidades minimizando o elemento de conexão (cabos, estradas, etc.).

Descoberta de árvores geradoras mínimas

Problema: dado um grafo simples com pesos nas arestas G , determinar uma árvore geradora mínima para ele.

Aplicações: conectar estações situadas em diferentes localidades minimizando o elemento de conexão (cabos, estradas, etc.).

Exemplo: conectar diversos computadores com o mínimo de cabos, estações de energia elétrica com o mínimo de fios, etc.

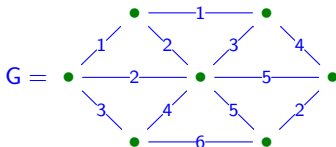
Descoberta de árvores geradoras mínimas

Problema: dado um grafo simples com pesos nas arestas G , determinar uma árvore geradora mínima para ele.

Aplicações: conectar estações situadas em diferentes localidades minimizando o elemento de conexão (cabos, estradas, etc.).

Exemplo: conectar diversos computadores com o mínimo de cabos, estações de energia elétrica com o mínimo de fios, etc.

Exercício: apresente uma árvore geradora mínima para o seguinte grafo:



Nota: existe AGM para um grafo simples com pesos nas arestas G sss G é conexo.

Lema sobre AGMs

Lema sobre AGMs

Lema: Seja $G = (V, E, w)$ um grafo simples conexo com pesos nas arestas, e $X \subseteq V$ um subconjunto de vértices. Seja $e \in E$ a aresta de menor custo entre algum nodo em X e algum nodo em $V - X$. Então, certamente e pertence a alguma AGM de G .

Lema sobre AGMs

Lema: Seja $G = (V, E, w)$ um grafo simples conexo com pesos nas arestas, e $X \subseteq V$ um subconjunto de vértices. Seja $e \in E$ a aresta de menor custo entre algum nodo em X e algum nodo em $V - X$. Então, certamente e pertence a alguma AGM de G .

Demonstração:

Assuma (por contradição) que uma AGM qualquer T não contém $e = \{v, w\}$.

Lema sobre AGMs

Lema: Seja $G = (V, E, w)$ um grafo simples conexo com pesos nas arestas, e $X \subseteq V$ um subconjunto de vértices. Seja $e \in E$ a aresta de menor custo entre algum nodo em X e algum nodo em $V - X$. Então, certamente e pertence a alguma AGM de G .

Demonstração:

Assuma (por contradição) que uma AGM qualquer T não contém $e = \{v, w\}$.

Como T é árvore, ela contém exatamente um caminho entre $v \in V$ e $w \in (V - X)$. Se adicionássemos e a T , teríamos um ciclo.

Lema sobre AGMs

Lema: Seja $G = (V, E, w)$ um grafo simples conexo com pesos nas arestas, e $X \subseteq V$ um subconjunto de vértices. Seja $e \in E$ a aresta de menor custo entre algum nodo em X e algum nodo em $V - X$. Então, certamente e pertence a alguma AGM de G .

Demonstração:

Assuma (por contradição) que uma AGM qualquer T não contém $e = \{v, w\}$.

Como T é árvore, ela contém exatamente um caminho entre $v \in V$ e $w \in (V - X)$. Se adicionássemos e a T , teríamos um ciclo.

Portanto, há na AGM uma aresta $f = \{v', w'\}$ tal que $v' \in V$ e $w' \in (V - X)$. Como e é de peso mínimo entre X e $V - X$, temos que $w(e) \leq w(f)$. Considere $T' = T - f + e$. Dois casos:

- $w(e) < w(f)$: temos que $W(T') < W(T)$, e portanto T não é AGM (contradição)
- $w(e) = w(f)$: temos que $W(T') = W(T)$, e portanto T' é AGM. Logo, há uma AGM contendo e (contradição).

Conteúdo

Grafos simples, incidência, adjacência
Isomorfismo, topologias comuns
Graus
Passeios
Conectividade
Operações sobre grafos
Grafos cíclicos, árvores e florestas
Grafos bipartidos e k-partidos
Pseudografos
Dígrafos
Grafos e dígrafos valorados

Busca em profundidade e busca em largura
Algoritmo de Dijkstra
Grafos eulerianos
Grafos hamiltonianos
Problema do caixeiro viajante
Árvores geradoras mínimas
Algoritmos de Prim e Kruskal
Grafos planares
Coloração de grafos
Enumeração de árvores
Coberturas de vértices
Emparelhamentos

Algoritmos para determinar AGMs

Veremos dois algoritmos para determinação de árvores geradoras mínimas para um grafo G :

- **Algoritmo de Prim**

Ideia: "crescer" uma árvore mínima a partir de um nodo, adicionando arestas de peso mínimo.

Algoritmos para determinar AGMs

Veremos dois algoritmos para determinação de árvores geradoras mínimas para um grafo G :

- **Algoritmo de Prim**

Ideia: "crescer" uma árvore mínima a partir de um nodo, adicionando arestas de peso mínimo.

- **Algoritmo de Kruskal**

Ideia: começar uma AGM a partir do grafo nulo e do conjunto de arestas a serem inseridas. Selecionar sempre a aresta de menor peso, adicionando-a ao grafo se ciclos não forem gerados.

Algoritmo de Prim

Entrada: grafo simples $G=(V,E,w)$ com pesos não-negativos nas arestas e conexo

Saída: árvore geradora mínima para G

Início

```
// inicialização
```

```
n <- escolha um nodo em V
```

```
T <- ({n},{})
```

```
// iteração
```

```
enquanto  $V(T) \neq V$  faça
```

```
    A <- conjunto de arestas entre  $V(T)$  e  $(V - V(T))$ 
```

```
     $\{x,y\}$  <- aresta de custo mínimo em A ( $x$  pertencendo a  $V(T)$ )
```

```
    T <- (  $V(T) \cup \{y\}$  ,  $E(T) \cup \{x,y\}$  )
```

```
// resultado
```

```
retorne T
```

Fim

Algoritmo de Kruskal

Entrada: grafo simples $G = (V, E, w)$ com pesos não-negativos nas arestas e conexo

Saída: árvore geradora mínima para G

Início

```
X ← conjunto de conjuntos unitários de nodos em V // componentes
Y ← E // conjunto de arestas a serem eliminadas
T ← (V, {}) // árvore a ser devolvida
cont ← 0 // contador de iteração
```

enquanto $\text{cont} < |V|$ faça

$\{a, b\} \leftarrow$ escolha a aresta de menor peso em Y

 se a e b estão em conjuntos diferentes A e B em X:

 então

$Y \leftarrow Y - \{\{a, b\}\}$ // remove aresta da lista

$T \leftarrow (V(T), E(T) \cup \{\{a, b\}\})$ // e a adiciona à árvore

 X ← atualize X unindo A e B // reduz componentes

$\text{cont} \leftarrow \text{cont} + 1$ // atualiza contagem

 senão

$Y \leftarrow Y - \{\{a, b\}\}$ // desconsidera aresta

 retorne T

Fim

Conteúdo

Grafos simples, incidência, adjacência

Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

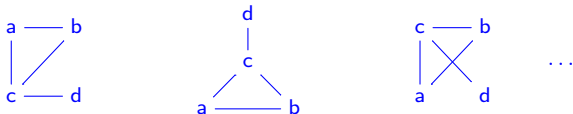
Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Imersões planares

Como já vimos, um mesmo grafo G pode ser desenhado de diversas formas (diversas representações gráficas)



Definição: uma associação

nodos $\mapsto$ pontos

arestas $\mapsto$ curvas simples contínuas entre os pontos associados a vértices

determina uma **imersão** (embedding) do grafo no plano X - Y .

Definição: Uma imersão é **planar** sss duas curvas distintas se encontram somente nos pontos dos vértices. Em inglês, uma imersão planar é chamada *plane graph*.

Grafos planares

Definição: um grafo G é **planar** sss G possui alguma imersão planar.

Intuitivamente: existe uma forma de desenhar ele no plano sem que tenhamos arestas cortando outras arestas.

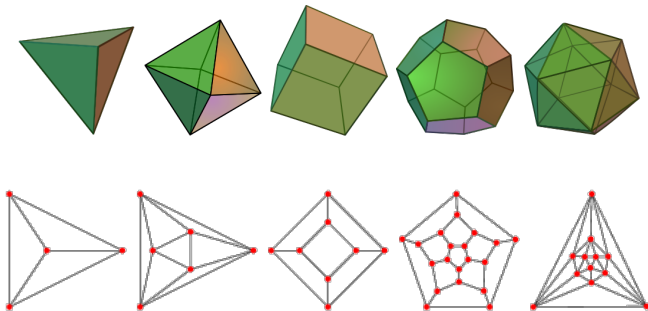
Exercício: determine quais dos seguintes grafos são planares:



Grafos planares: exemplos

Os grafos que representam os cinco poliedros regulares são planares:

Ideia: “expandir” um do lados do poliedro, achatando a superfície tridimensional. Considera-se a região externa a “face” do lado expandido.



Grafos planares: definições de topologia

Definição: um **conjunto aberto** (no plano) é um subconjunto $X \subseteq \mathbb{R}^2$ tal que, para qualquer ponto $p \in X$ há uma dada distância ϵ para a qual todos os pontos no plano a ϵ de distância de p estão em X .

Definição: uma **região** R é um conjunto aberto (no plano) onde, para qualquer par de pontos p_1 e p_2 , há um caminho simples contínuo entre eles em R .

Definição: uma **face** de uma imersão de um grafo G no plano é uma região maximal que não contém nenhum ponto da imersão.

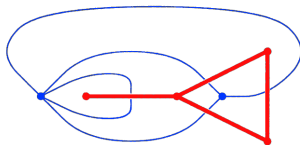
Teorema: (Jordan) uma curva c em $\mathbb{R}^2$ fechada contínua e simples (não repete elementos, com exceção dos pontos iniciais/finais) divide o plano em exatas duas regiões: interna e externa, sendo que toda curva contínua ligando um ponto interno a um ponto externo intersecciona c em no mínimo um ponto.

Grafos planares: grafo dual planar

Definição: seja G um pseudografo planar. O **pseudografo dual planar** de uma **imersão planar de G** , também chamado dual geométrico, é o pseudografo $G' = (V', E')$ onde

- V' é o conjunto de faces da imersão planar de G .
- existe uma aresta $\{f_1, f_2\} \in E'$ para cada aresta $e \in E$ dividindo as faces f_1 e f_2 em G .

Exemplo: pseudografo G e o dual de G .



Nota: duas imersões planares distintas de um mesmo pseudografo G podem ter duais não-isomorfos.

Grafos planares: fórmula de Euler (lema)

Lema: Para um grafo planar G ,

$$v + f = e + c + 1$$

onde

- v : número de vértices $= |V|$
- f : número de faces de uma imersão planar de G
- e : número de arestas $= |E|$
- c : número de componentes conexos

Grafos planares: fórmula de Euler (prova do lema)

Demonstração: por indução no número de arestas:

- caso base $e = 0$: $v + 1 = 0 + v + 1$ (prova válida)
- passo indutivo: suponha que $v + f = e + c + 1$ valha para grafo planar G (hipótese indutiva). Tome uma imersão planar de G , como base, e considere a adição de uma aresta i entre dois nós não-adjacentes de G de tal forma que não haja cruzamento de arestas. Duas possibilidades:
 - i conecta dois nós de um mesmo componente conexo. Nesse caso, aumenta-se em um o número de faces (planaridade) e o número de arestas, onde temos $v + (f + 1) = (e + 1) + c + 1$.
 - i conecta dois nós em componentes conexos distintos. Neste caso, não se adicionam faces porém reduz-se (em uma unidade) o número de componentes conexos do grafo, e temos $v + f = (e + 1) + (c - 1) + 1$.

Grafos planares: fórmula de Euler

Teorema: Para um grafo planar e conexo G ,

$$v + f = e + 2$$

onde

- v : número de vértices
- f : número de faces de uma imersão planar de G
- e : número de arestas

Demonstração: usando a prova do lema anterior, contando $v - 1$ vezes o caso 2 (conexão de nodos em componentes conexos distintos).

Grafos planares: quantidade máxima de arestas

Os grafos planares possuem a propriedade de serem esparsos (poucas arestas)

Definição: Um grafo simples planar $G = (V, E)$ é **maximal** se, para toda aresta $e \notin E$, $G' = (V, E \cup \{e\})$ não é planar.

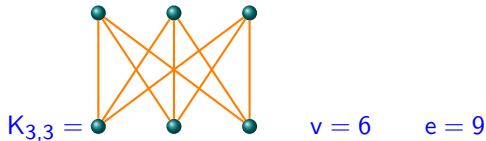
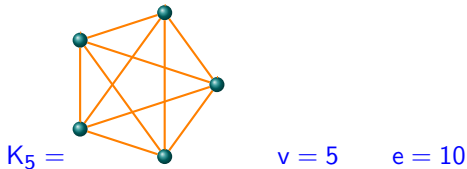
Teorema: Um grafo simples planar maximal G possui $e = 3v - 6$ arestas.

Demonstração: 1. G é conexo. 2. todas as regiões de um grafo simples planar maximal são triângulos. 3. Como cada face é composta por três arestas, podemos inicialmente aproximar $3f \propto e$. 4. contudo, isso estaria contando duas vezes cada arestas, portanto a fórmula exata é $3f = 2e$. 5. substituindo f na fórmula de Euler pelo resultado anterior, obtemos $e = 3v - 6$.

Corolário: se G é planar, então $e \leq 3v - 6$.

Não-planaridade de $K_{3,3}$ e K_5

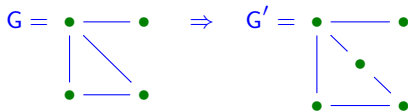
Os seguintes grafos não-planares são especiais:



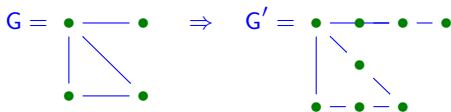
Caracterização de planaridade

Definição: Dado um grafo simples G , uma **extensão direta** de G é um grafo G' obtido pela eliminação de uma aresta em G e a introdução de um caminho de tamanho 2 no lugar da aresta removida (com um nodo novo).

Exemplo:



Definição: Dado um grafo simples G , uma **extensão** de G é um grafo G' obtido através de um número finito (possivelmente zero) de sucessivas extensões diretas de G .

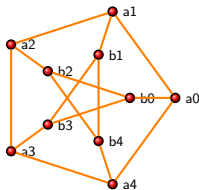


Caracterização de grafos planares

Teorema: (Kuratowski) um grafo simples é não-planar sss possui como subgrafo uma extensão de K_5 ou de $K_{3,3}$

Demonstração: não será vista nessa disciplina.

Exemplo: mostre que o grafo de Petersen não é planar utilizando o teorema de Kuratowski.



$\Rightarrow$

$$\begin{aligned} V_1 &= \{a1, b0, a4\} \\ V_2 &= \{a0, b2, b3\} \end{aligned}$$

O grafo de Petersen com as arestas $\{a2, a3\}$ e $\{b1, b4\}$ removidas é uma extensão de $K_{3,3}$, onde V_1 e V_2 representam os dois conjuntos de vértices.

Caracterização de grafos planares (2)

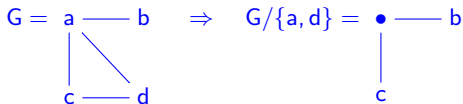
Definição: Contração de aresta (grafos simples).

Seja $G = (V, E)$ um grafo simples e uma aresta $e = \{v, w\} \in E$. Definimos a contração de e em G , denotada G/e , como o grafo $G' = (V', E')$ onde

$$V' = V - \{v, w\} \uplus \{\bullet\}$$

$$E' = \{e \mid e \in E, v \notin e, w \notin e\} \cup \{\{\bullet, x\} \mid x \in V \wedge (v \text{ adj } x \vee w \text{ adj } x)\}$$

Exemplo:

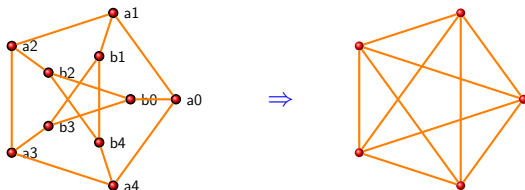


Teorema: (Wagner) um grafo simples G é não-planar sss possui um subgrafo que pode ser transformado em K_5 ou $K_{3,3}$ através de um número finito (possivelmente zero) de contrações de aresta.

Demonstração: não será vista nessa disciplina.

Caracterização de grafos planares (2)

Exemplo: mostre que o grafo de Petersen não é planar utilizando o teorema de Wagner.



Contrações: $/\{a_0, b_0\}/\{a_1, b_1\}/\{a_2, b_2\}/\{a_3, b_3\}/\{a_4, b_4\}$

Conteúdo

Grafos simples, incidência, adjacência

Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Coloração de grafos

Definição: uma **coloração** (de vértices) de um grafo simples $G = (V, E)$ é uma função $f : V \rightarrow C$, onde, para todo $x, y \in V$,

$$\{x, y\} \in E \Rightarrow f(x) \neq f(y)$$

Cada elemento do conjunto C é denominado uma cor.

Definição: $|\text{img}(f)|$ é o **número de cores** de f .

Definição: um grafo G é **k-colorível** sss existe uma coloração de G com no máximo k cores.

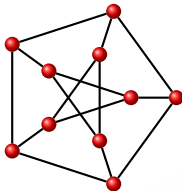
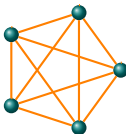
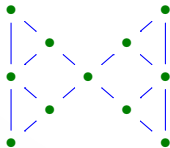
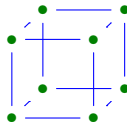
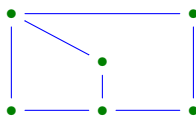
Nota: um grafo é **k-colorível** sss for **k-partido**.

Coloração de grafos: número cromático

Definição: o número cromático de G é o menor número de cores k tal que G seja k -colorível.

Notação: denotamos of número cromático de G por $\chi(G)$. Dizemos que G é k -cromático sss seu número cromático é k .

Exercício: determine o número cromático dos grafos abaixo:



Coloração de grafos: exercício (2)

Determine os números cromáticos das seguintes famílias de grafos:

- nulo
- completo
- caminho
- círculo
- grade
- estrela
- roda
- k-cubo

Coloração de grafos: alguns resultados

Alguns resultados sobre o número cromático $\chi(G)$ de um grafo simples $G = (V, E)$

- $\chi(G) \leq |V|$ (trivial)
- $\omega(G) \leq \chi(G)$, onde $\omega(G)$ é o número de nodos do maior clique de G . (trivial)
- se $|E| = 0$, então $\chi(G) \leq 1$. (trivial)
- se G bipartido, então $\chi(G) \leq 2$. (trivial)
- $\chi(G) \leq \Delta(G) + 1$. teorema, a ser visto
- se G planar, então $\chi(G) \leq 5$ teorema, a ser visto
- se G planar, então $\chi(G) \leq 4$ teorema, a ser visto (sem prova)

Coloração de grafos: limites para $\chi(G)$

Teorema: Em um grafo simples $G = (V, E)$ temos $\chi(G) \leq \Delta(G) + 1$

Intuição: o número cromático é sempre menor ou igual ao maior grau do grafo mais um.

Demonstração: seja $v_1, v_2, \dots, v_n$ uma enumeração qualquer dos vértices de V . Seja $C = c_1, \dots, c_k$ uma coleção de $k = \Delta(G) + 1$ cores. Para cada v_i atribua a primeira cor da enumeração C que não ocorre em nenhum dos $v_1, \dots, v_{i-1}$ anteriores que são adjacentes a v_i . No pior caso, o número máximo de vizinhos adjacentes que ocorrem antes de v_i é $\Delta(G)$, e portanto precisamos de uma cor adicional para tal nodo c_k .

Coloração de grafos: limites para $\chi(G)$ (2)

Lema: se o grafo simples $G = (V, E)$ é planar, então existe um vértice v tal que $\deg(v) \leq 5$

Demonstração: . Por contradição, assuma que $\deg(x) \geq 6$ para todo $x \in V$. Como G é planar, a fórmula de Euler vale, e temos

$$|E| \leq 3|V| - 6$$

Contudo, como temos que todo $x \in V$ tem grau 6 ou mais, temos que

$$|E| \geq \frac{1}{2} \times 6|V| = 3|V|$$

o que gera, obviamente, uma contradição.

Logo, existe no mínimo um vértice $x \in V$ tal que $\deg(x) \leq 5$.

Coloração de grafos: limites para $\chi(G)$ (3)

Teorema: se G é simples e planar, então $\chi(G) \leq 5$.

Demonstração: por indução no número de vértices de G .

Coloração de grafos: limites para $\chi(G)$ (4)

Teorema: se G é simples e planar, então $\chi(G) \leq 4$.

Demonstração: (Appel & Haken, 1976) não será vista nessa disciplina.

Problema famoso de teoria dos grafos, foi durante antes uma conjectura confirmada por exemplos, porém sem prova.

Primeiro teorema importante cuja prova recebeu ajuda de computadores na verificação (por força bruta) de um grande número de subcasos.

Conteúdo

Grafos simples, incidência, adjacência

Isomorfismo, topologias comuns

Graus

Passeios

Conectividade

Operações sobre grafos

Grafos cíclicos, árvores e florestas

Grafos bipartidos e k-partidos

Pseudografos

Dígrafos

Grafos e dígrafos valorados

Busca em profundidade e busca em largura

Algoritmo de Dijkstra

Grafos eulerianos

Grafos hamiltonianos

Problema do caixeiro viajante

Árvores geradoras mínimas

Algoritmos de Prim e Kruskal

Grafos planares

Coloração de grafos

Enumeração de árvores

Coberturas de vértices

Emparelhamentos

Enumeração de árvores

Pergunta: quantas árvores existem com exatos n vértices?

Enumeração de árvores

Pergunta: quantas árvores existem com exatos n vértices?

Nota: Árvores concretas, não classes de isomorfismo.

Podemos começar a contagem com número pequeno de vértices:

Enumeração de árvores

Pergunta: quantas árvores existem com exatos n vértices?

Nota: Árvores concretas, não classes de isomorfismo.

Podemos começar a contagem com número pequeno de vértices:

$ V $	1	2	3	4	5	...
número de árvores	1	1	3	16	125	...

Pode-se detectar algum padrão?

Enumeração de árvores: fórmula de Cayley

Teorema: (Cayley, 1889) o número de árvores com n vértices é n^{n-2} .

Enumeração de árvores: fórmula de Cayley

Teorema: (Cayley, 1889) o número de árvores com n vértices é n^{n-2} .

Em 1918, Prüfer apresentou uma prova alternativa desse teorema.

Enumeração de árvores: fórmula de Cayley

Teorema: (Cayley, 1889) o número de árvores com n vértices é n^{n-2} .

Em 1918, Prüfer apresentou uma prova alternativa desse teorema.

A prova de Prüfer é interessante por introduzir uma codificação de árvores conhecida como código de Prüfer.

Enumeração de árvores: fórmula de Cayley

Teorema: (Cayley, 1889) o número de árvores com n vértices é n^{n-2} .

Em 1918, Prüfer apresentou uma prova alternativa desse teorema.

A prova de Prüfer é interessante por introduzir uma codificação de árvores conhecida como código de Prüfer.

Ideia: estabelecer uma bijeção:

árvores com n vértices $\Leftrightarrow$ strings de tamanho $n - 2$ sobre alfabeto $\{1, \dots, n\}$

Código Prüfer: codificação

Entrada: árvore T com conjunto de vértices $V \subseteq \mathbb{N}$.

Saída: string de tamanho $n-2$ sobre alfabeto $\{1, \dots, n\}$, onde $n = |V|$.

Início

```
// Inicializa variáveis
```

```
a <- novo vetor com posições 1 ... n-2
```

```
i <- 1
```

```
// Iteração
```

```
enquanto i <= n-2 faça
```

```
    f <- folha de menor número em T
```

```
    g <- vértice conectado a f
```

```
    a[i] <- g
```

```
    T <- T - f
```

```
    i <- i + 1
```

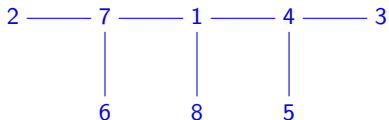
```
// Finalização
```

```
retorna a
```

Fim.

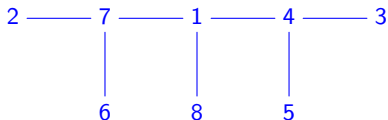
Código Prüfer: exemplo de codificação

Árvore:



Código Prüfer: exemplo de codificação

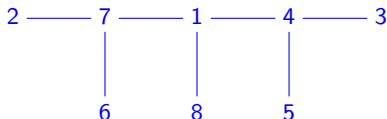
Árvore:



Código Prüfer: 744171

Código Prüfer: exemplo de codificação

Árvore:



Código Prüfer: 744171

Observações

- o código não faz nenhuma referência às folhas da árvore.
- todos os nodos não-folha ocorrem no código.
- o primeiro nodo "removido" no processo de codificação é o menor número que não ocorre no código.

Código Prüfer: decodificação

Entrada: string s de tamanho $n - 2$ sobre alfabeto $\{1, \dots, n\}$

Saída: árvore de n vértices com conjunto de vértices $\{1, \dots, n\}$

Início

```
// Inicializa variáveis
```

```
i <- 1
```

```
T <- grafo nulo com vértices numerados de 1 a n
```

```
// Iteração
```

```
enquanto i <= n-2 faça
```

```
    f <- menor número que não ocorre em s
```

```
    T <- T U {f,s[i]} // adiciona aresta
```

```
    s[i] <- f // substitui posição
```

```
    i <- i + 1
```

```
// Finalização
```

```
{f,g} <- dois números que não ocorrem em s
```

```
T <- (V(T), E(T) U {{f,g}})
```

```
retorna T
```

Fim.

Código Prüfer: exercício de decodificação

Exercício: decodifique a árvore com 10 vértices (dígitos 1 a 10) representada pela seguinte string:

2, 3, 9, 5, 1, 8, 5, 2

Código Prüfer: prova de bijeção

Objetivo: mostrar que, para cada

$$a = (a_1, \dots, a_n) \in \{1, \dots, n\}^{n-2}$$

há exatamente uma árvore T com conjunto de vértices $\{1, \dots, n\}$ onde $\text{Prufer}^{-1}(a) = T$.

Prova: indução no tamanho de a .

- **caso base** (string vazia) $\Rightarrow$ exatamente uma única árvore $T = 1 \text{ --- } 2$.
- **passo indutivo** ($a = x, a'$). Primeira folha deletada na construção de a é o menor número que não ocorre na árvore original (vamos chamá-lo f). A folha f está conectada a x em T . Pela hipótese indutiva, existe exatamente uma única árvore T' tal que $\text{Prufer}^{-1}(a') = T'$ (desconsiderando f como possível vértice). Tomamos $T = T' \cup \{f, x\}$. □

Enumeração de árvores geradoras

Podemos enxergar a aplicação da fórmula de Cayley como o cálculo do *número de árvores geradoras* de um grafo completo com n vértices.

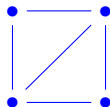
No caso geral, contagem de árvores geradoras em grafos arbitrários não é tão simples, muitas vezes não havendo fórmulas fechadas.

Contudo, há um método de cálculo recorrente que pode ser utilizado para determinar tal número em um grafo arbitrário.

Intuição: selecionar uma aresta, e contar separadamente as árvores geradoras que passam por aquela aresta e as que não passam.

Enumeração de árvores geradoras

Exemplo:



Esse grafo possui 4 árvores geradoras passando pela diagonal, e 4 árvores evitando a diagonal.

- contagem evitando a diagonal: basta remover a aresta, e contar as árvores geradoras do restante do grafo.
- contagem contando a diagonal: vamos utilizar a operação **contração de aresta em multigrafo**, e a partir daí contar as árvores geradoras.

Enumeração de árvores geradoras (2)

Definição: seja G um pseudografo. Uma **contração de aresta em multigrafo** de uma aresta $e = \{x, y\}$ em G , denotada $(G \cdot e)$, consiste do pseudografo obtido pela remoção da aresta e e da identificação dos vértices x e y (mantendo todas as arestas incidentes aos vértices x e y sobre esse novo vértice).

Exemplo:



Importante: a operação de contração só é possível quando não são gerados loops, isto é, quando não há arestas paralelas à aresta a ser contraída.

Enumeração de árvores geradoras (3)

Teorema: denotemos por $\tau(G)$ o número de árvores geradoras de um multigrafo G . Se $(G \cdot e)$ é definido (não gera laço), temos

$$\tau(G) = \tau(G - e) + \tau(G \cdot e)$$

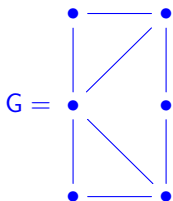
Demonstração: mostrar que a contração define uma bijeção entre árvores geradoras de $(G \cdot e)$ e árvores geradoras de G que passam por e .

Nota: casos base:

- grafo desconexo (0 árvores geradoras).
- grafos “multiárvore” (multigrafo que se torna árvore se eliminarmos arestas paralelas).

Enumeração de árvores geradoras: exercício

Exercício: determine o número de árvores geradoras do seguinte grafo, calculando através do método recorrente visto.



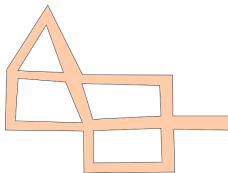
Conteúdo

Grafos simples, incidência, adjacência
Isomorfismo, topologias comuns
Graus
Passeios
Conectividade
Operações sobre grafos
Grafos cíclicos, árvores e florestas
Grafos bipartidos e k-partidos
Pseudografos
Dígrafos
Grafos e dígrafos valorados

Busca em profundidade e busca em largura
Algoritmo de Dijkstra
Grafos eulerianos
Grafos hamiltonianos
Problema do caixeiro viajante
Árvores geradoras mínimas
Algoritmos de Prim e Kruskal
Grafos planares
Coloração de grafos
Enumeração de árvores
Coberturas de vértices
Emparelhamentos

Problema das câmeras

Suponha um museu (ou galeria de arte) onde estão expostos diversos quadros muito valiosos. Para elaborar um sistema de monitoramento adequado, há disponíveis diversas câmeras de vigilância, que podem ser instaladas em entroncamentos de corredores do museu. Por simplicidade, assumimos que cada câmera permite a vigilância (em 360 graus) de todos os corredores a ela associados **até o próximo entroncamento**.

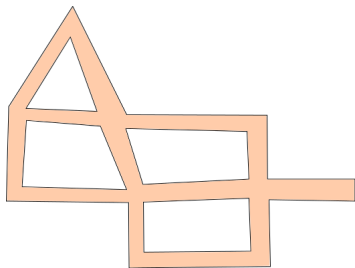


Mapa do museu

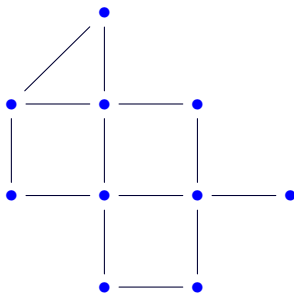
Pergunta: qual o menor número de câmeras que precisamos instalar (e onde elas devem estar) para que não haja nenhum corredor do museu descoberto?

Problema das câmeras: grafo associado

Podemos a partir do mapa do museu, gerar um grafo associado:



Mapa do museu



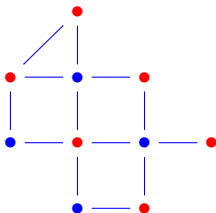
Grafo associado

Cobertura de vértices

Definição: Seja $G = (V, E)$ um grafo simples. Uma **cobertura de vértices** é um subconjunto $X \subseteq V$ tal que, para toda aresta $\{x, y\} \in E$,

$$x \in X \quad \text{ou} \quad y \in X$$

Exemplo: abaixo temos em **vermelho** uma cobertura de vértices para o grafo do museu.

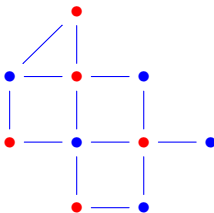


Cobertura de vértices mínima

Definição: uma cobertura de vértices X de um grafo simples G é **mínima** sss para toda cobertura de vértices Y de G , $|X| \leq |Y|$.

Notação: denotamos $\beta(G) = |X|$ a cardinalidade de uma cobertura de vértices mínima X de G .

Exemplo: no caso do museu, temos abaixo (**em vermelho**) uma cobertura mínima com 5 nodos (note que há outras coberturas mínimas possíveis).



$$\beta(G) = 5$$

Cobertura de vértices: algoritmos

Determinar para um grafo de entrada G uma cobertura mínima é o problema da Cobertura Mínima de Vértices (CMV), em inglês *minimum vertex cover (MVC)*.

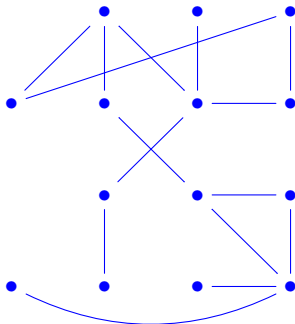
Algoritmo ingênuo (ineficiente): gerar progressivamente todos os subconjuntos de vértices (ordenados por tamanho) e testar se todas as arestas estão cobertas.

CMV é um problema conhecidamente NP-difícil, para o qual habitualmente são utilizadas heurísticas (ou métodos exatos otimizados).

Outras disciplinas detalharão algoritmos e heurísticas para esse problema.

Exercício:

Determine uma cobertura de vértices mínima e parâmetro β do seguinte grafo:

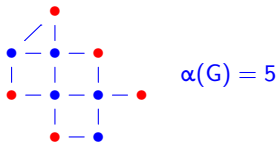


Maior conjunto independente

Relembrando: um *conjunto independente* de um grafo simples $G = (V, E)$ é um subconjunto de vértices $X \subseteq V$ tal que, para quaisquer dois nodos $x, y \in X$, não há aresta entre eles ($\{x, y\} \notin E$).

Notação: denotamos $\alpha(G)$ o tamanho do conjunto independente máximo (de maior tamanho) de G . Também chamado *número de estabilidade* de G .

Exemplo: para o grafo do museu, **em vermelho** temos um conjunto independente máximo.



Pergunta: dado um grafo simples G , existe alguma relação entre $\alpha(G)$ e $\beta(G)$?

Relação entre $\alpha(G)$ e $\beta(G)$

Seja $G = (V, E)$ um grafo simples.

Nota: se $X \subseteq V$ for uma cobertura de vértices, então $V - X$ é um conjunto independente.

Nota: se $Y \subseteq V$ for um conjunto independente, então $V - Y$ é uma cobertura de vértices.

Lema:

$$\alpha(G) + \beta(G) = |V|$$

Conteúdo

Grafos simples, incidência, adjacência
Isomorfismo, topologias comuns
Graus
Passeios
Conectividade
Operações sobre grafos
Grafos cíclicos, árvores e florestas
Grafos bipartidos e k-partidos
Pseudografos
Dígrafos
Grafos e dígrafos valorados

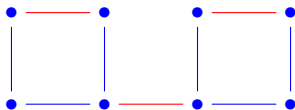
Busca em profundidade e busca em largura
Algoritmo de Dijkstra
Grafos eulerianos
Grafos hamiltonianos
Problema do caixeiro viajante
Árvores geradoras mínimas
Algoritmos de Prim e Kruskal
Grafos planares
Coloração de grafos
Enumeração de árvores
Coberturas de vértices
Emparelhamentos

Emparelhamentos

Definição: seja $G = (V, E)$ um grafo simples. Um **emparelhamento** de G é um subconjunto $A \subseteq E$ onde, para todas arestas $e_1, e_2 \in A$, temos que e_1 e e_2 não são *adjacentes* (isto é, não possuem nenhum vértice em comum).

Nota: um emparelhamento também é chamado *acoplamento*, *conjunto independente de arestas* ou *casamento*. Em inglês, usa-se a expressão *matching*.

Exemplo: emparelhamento (em vermelho) contendo três arestas.



Emparelhamentos perfeitos

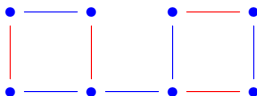
Definição: se um emparelhamento A de G incidir sobre todos os vértices do grafo, dizemos que A é um **emparelhamento perfeito**

Nota: só é possível haver emparelhamentos perfeitos quando $|V|$ for par.

Notação: denotamos $\alpha'(G)$ o tamanho do emparelhamento máximo (de maior tamanho) do grafo G

Nota: note que o maior emparelhamento pode não ser perfeito.

Exemplo: emparelhamento perfeito e máximo (em vermelho):

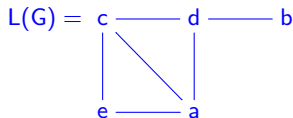
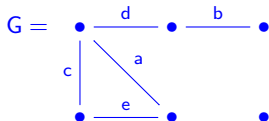


Emparelhamentos e conjuntos independentes

Definição: seja $G = (V, E)$ um grafo simples. Denotamos **grafo linha de G** o grafo simples $L(G) = (V', E')$ onde

- $V' = E$
- $\{e_1, e_2\} \in E'$ sss as arestas e_1 e e_2 compartilham exatamente um vértice.

Exemplo:

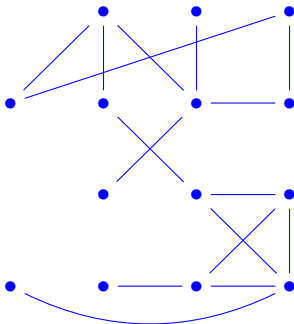


Lema: como cada emparelhamento em G é um conjunto independente em $L(G)$, temos

$$\alpha'(G) = \alpha(L(G))$$

Exercício:

Determine um emparelhamento máximo para o seguinte grafo:



O emparelhamento encontrado é perfeito?